COMPUTER ORGANIZATION · HIT

计算机组成原理

哈尔滨工业大学 刘宏伟 · 135 个视频速学笔记

抽帧读图，把 32.9 小时的课程压成一份可扫读的骨架：

每节一句主旨 · 3–6 个关键点 · 一个易错考点

哈尔滨工业大学 · 刘宏伟 · 135 个视频

目　录

01 这门课在讲什么

02 课程地图

03 第 1 章 计算机系统概论

04 第 2 章 计算机的发展及应用

05 第 3 章 系统总线

06 第 4 章 存储器

07 第 5 章 输入输出系统

08 第 6 章 计算机的运算方法

09 第 7 章 指令系统

10 第 8 章 CPU 的结构与功能

11 第 9 章 控制单元的功能

12 第 10 章 控制单元的设计

> 来源：`~/Documents/资料整理/学习/学习/学习视频/计算机组成原理-哈工大`
> 覆盖：135 个视频，共 32.9 小时
> 做法：逐视频均匀抽帧、去重后读图，提取幻灯片要点整理成笔记（不含语音转写）

这门课在讲什么

135 讲、33 小时，是**国内考研 408 与期末考的标准底本**——内容严格对齐唐朔飞《计算机组成原理》（第 2 版）的十章结构。刘宏伟的讲法偏「板书推导 + 逐条拆解」：每个部件都从「为什么需要它」讲到「它由什么构成」再到「它怎么工作」，中间穿插大量例题与真值表。

这门课的学习曲线在第 4 章（存储器）和第 6 章（运算方法）会突然变陡：前者要记一堆芯片引脚与扩展方式，后者要把补码乘除的每一步移位都算对。**建议顺序：第 1、3 章建立框架 → 第 4、5 章攻硬件结构 → 第 6 章死磕运算 → 第 7—10 章串成一台完整的 CPU。**第 2 章（发展史）可以先跳过。

课程地图

板块	讲次	时长	一句话
第 1 章 计算机系统概论	第 1 章	2.9 h	层次结构、冯·诺依曼、五大部件、技术指标
第 2 章 计算机的发展及应用	第 2 章	0.8 h	发展史、摩尔定律、应用领域
第 3 章 系统总线	第 3 章	1.6 h	总线分类、性能指标、仲裁与通信控制
第 4 章 存储器	第 4 章	7.2 h	层次结构、SRAM/DRAM、芯片扩展、Cache、辅存
第 5 章 输入输出系统	第 5 章	3.8 h	I/O 接口、程序查询、中断、DMA
第 6 章 计算机的运算方法	第 6 章	7.4 h	补码、定点乘除、浮点、ALU
第 7 章 指令系统	第 7 章	2.6 h	指令格式、寻址方式、RISC/CISC
第 8 章 CPU 的结构与功能	第 8 章	2.8 h	CPU 结构、指令周期、流水线、中断系统
第 9 章 控制单元的功能	第 9 章	1.4 h	微操作命令、时序、同步/异步控制
第 10 章 控制单元的设计	第 10 章	2.5 h	组合逻辑设计、微程序设计
合计	135 个视频	**32.9 h**	—

第 1 章 计算机系统概论

> 全书的概念地基：先把「体系结构 vs 组成」分清，再记住冯·诺依曼机的四条特征和五大部件，最后掌握字长、MIPS、CPI 这些量化指标。

1.1 计算机系统简介1.1-b（18.6 分钟）

- **主旨**：讲计算机系统的层次结构与"体系结构 vs 组成"的区别，是全书的概念总纲。
- **关键点**：
  1. 管理系统复杂性的第一招是抽象（abstraction）：有目的地隐藏过程或制品的某些细节，只保留本质，从而突出主要方面。
  2. 层次结构自下而上为：微指令系统→微程序机器 M0、机器语言→实际机器 M1、操作系统→虚拟机器、汇编语言→虚拟机器 M2、高级语言→虚拟机器 M3。
  3. 层间转换方式各不相同：M3 由编译程序译成汇编、M2 由汇编程序译成机器语言、操作系统用机器语言解释、M1 用微指令解释、M0 由硬件直接执行微指令。
  4. 计算机体系结构（computer architecture）是程序员所见到的属性与概念性的结构与功能特性，如指令系统、数据类型、寻址技术、I/O 机理。
  5. 计算机组成（computer organization）是实现这些属性的具体实现，例如"有无乘法指令"属体系结构，"如何实现乘法指令"属组成。
- **易错/考点**：判断"某特性属于体系结构还是组成"——面向程序员可见的归体系结构，面向具体实现的归组成。

1.1 计算机系统简介 1.1-a（17.8 分钟）

- **主旨**：用"多态性"和 TOP500 榜单说明现代计算机的谱系，并给出软硬件的划分。
- **关键点**：
  1. 现代计算机呈多态性，从大到小为 HPC、Mainframe（大型机）、Server、PC/Client、Laptop、Info. appliances（信息家电）、Sensors（传感器）。
  2. 物联网的实质是把感应器嵌入装备到电网、铁路、桥梁、隧道、公路、建筑、供水系统、大坝、油气管道等物体中并普遍连接，再与互联网整合，形成"智慧地球"。
  3. 截至 2014 年 11 月 TOP500 第一为中国的天河二号（Tianhe-2），Rmax 33 862.7 TFlop/s、Rpeak 54 902.4 TFlop/s，18 000 节点级规模、功耗 17 808 kW。
  4. "泰坦"（Titan-Cray XK7）有 200 个机柜、18 688 个计算节点，每节点一颗 AMD Opteron 6200 十六核 CPU 加一块 NVIDIA Tesla K20 GPU 加速卡，峰值 20 PFlops 以上，可扩展到 50 PFlops。

5. 计算机系统 = 硬件 + 软件：硬件是计算机实体（主机、外设等），软件分系统软件与应用软件。

- **易错/考点**：系统软件包含语言处理程序、操作系统、服务性程序、数据库管理系统、网络软件——操作系统只是其中一类。

1.1 计算机系统简介课程简介（19.9 分钟）

- **主旨**：交代本课程的讲授内容、教材、在课程体系中的位置以及全书结构。
- **关键点**：

1. 讲授内容三条主线：基本部件的结构和组织方式、基本运算的操作原理、基本部件和单元的设计思想。
2. 主教材为唐朔飞《计算机组成原理》（第 2 版），参考教材为 Hennessy & Patterson 的 Computer Organization & Design 与 Harris 的 Digital Design and Computer Architecture。
3. 课程体系纵向自顶向下：Application→Compiler/Operating System→Instruction Set Architecture→Processor/Memory/IO System→Datapath & Control→Digital Design→Circuit Design→Transistors。
4. 本课程定位在 ISA 之下的"如何实现"：基本逻辑单元、处理器基本知识，上承计算机系统结构、下接数字逻辑。
5. 全书分篇：第 2 篇计算机硬件结构（存储器、I/O、系统总线、CPU）、第 4 篇 CU（ALU、CU、寄存器、控制存储器）。

- **易错/考点**：课程内容按"机器—数字—CPU"三块整合：基本概念/总线/存储器/IO、数的表示与运算器、指令系统与控制器。

1.2 计算机的基本组成1.2-a1（15.6 分钟）

- **主旨**：列出冯·诺依曼计算机的四条特点并给出五大部件框图。
- **关键点**：

1. 计算机由五大部件组成：运算器、控制器、存储器、输入设备、输出设备。
2. 指令和数据以同等地位存于存储器，可按地址寻访——这是"存储程序"的核心。
3. 指令和数据都用二进制表示，指令由操作码和地址码两部分组成。
4. 冯·诺依曼硬件框图中，运算器居中，存储器与输入/输出设备分列两侧，控制器发出控制信号协调各部。

- **易错/考点**：五大部件的说法要背准；"指令与数据同等地位存放、按地址访问"是冯·诺依曼机区别于其他结构的标志。

1.2 计算机的基本组成1.2-a2（8.5 分钟）

- **主旨**：对比以存储器为中心的旧框图与现代计算机硬件框图，引入层次化设计思想。
- **关键点**：
  1. 以存储器为中心的框图：输入设备把数据和程序送入存储器，运算器从存储器取数、结果再写回，输出设备取计算结果，控制器统管。
  2. 现代框图把运算器与控制器合为 CPU（运算器含 ALU，控制器含 CU），CPU 加主存构成主机。
  3. 存储器分为主存与辅存，输入/输出设备统称 I/O 设备，主机加 I/O 设备即硬件。
  4. 管理复杂性的第二招是层次化（Hierarchy）：把被设计的系统划分为多个模块或子模块。
- **易错/考点**：主机 = CPU + 主存，不含辅存与 I/O；ALU 属运算器、CU 属控制器，不可混淆。

1.2 计算机的基本组成1.2-b（19.4 分钟）

- **主旨**：用一个二次多项式的编程实例，说明从问题到指令序列再到指令格式的完整过程。
- **关键点**：
  1. 用计算机解题先要做上机前准备：建立数学模型（如 $u = Um \cdot \sin\omega t$）、确定计算方法。
  2. 编程举例计算 ax^2+bx+c，为减少运算可改写为 $(ax+b)x+c$。
  3. 指令格式举例：操作码 000001 表示"取数"，地址码 0000001000 指向 α，语义记为 [α]→ACC。
  4. 常用指令语义记法：存数 [ACC]→β、加 [ACC]+[γ]→ACC、乘 [ACC]×[δ]→ACC、打印 [σ]→打印机、停机。
  5. 程序清单中指令与数据同处一个存储器：地址 0~6 存指令，地址 8~12 依次存原始数据 x、a、b、c 与结果。
- **易错/考点**：方括号 [x] 表示"取 x 地址中的内容"，与 x 本身（地址）严格区分。

1.2 计算机的基本组成1.2-c（9.1 分钟）

- **主旨**：剖析主存储器的内部层次与 MAR/MDR 的作用。
- **关键点**：
  1. 层次类比：存储体—存储单元—存储元件，对应大楼—房间—床位（有人/无人即 1/0）。
  2. 存储单元存放一串二进制代码，这串代码称存储字（word），其位数称存储字长。
  3. 每个存储单元赋予一个地址，按地址访问是存储器工作的基本方式。
  4. MAR（Memory Address Register，存储器地址寄存器）位数反映存储单元的个数。
  5. MDR（Memory Data Register，存储器数据寄存器）位数反映存储字长；若 MAR=4 位、MDR=8 位，则共有 2^4 个单元、每单元 8 位。

- **易错/考点**：MAR 位数决定"有多少个单元"，MDR 位数决定"每个单元多宽"，两者不可互换。

1.2 计算机的基本组成1.2-d（22.0 分钟）

- **主旨**：讲运算器的 ACC/MQ/X/ALU 结构，并逐一演示加、减、乘、除的操作过程。
- **关键点**：
  1. 运算器由 ACC（累加器）、MQ（乘商寄存器）、X（操作数寄存器）与 ALU 组成，ACC 与 MQ 之间可直接传送。
  2. 加法：初态 ACC 存被加数，指令"加 M"执行 [ACC]+[X]→ACC，和留在 ACC。
  3. 减法：初态 ACC 存被减数，指令"减 M"执行 [ACC]−[X]→ACC。
  4. 乘法：初态 ACC 存被乘数，先 [M]→MQ、[ACC]→X、0→ACC，再 [X]×[MQ]→ACC//MQ，高位积在 ACC、低位积在 MQ。
  5. 除法：初态 ACC 存被除数，[M]→X 后执行 [ACC]÷[X]→MQ，商在 MQ，余数留在 ACC。
- **易错/考点**：乘法与除法的结果寄存器分配相反——乘"积在 ACC/MQ"，除"商在 MQ、余数在 ACC"。

1.2 计算机的基本组成1.2-e（21.9 分钟）

- **主旨**：讲控制器的组成与取指/分析/执行三阶段，串起一条指令的完整执行流程。
- **关键点**：
  1. 控制器由 CU（控制单元）、IR（指令寄存器）、PC（程序计数器）组成，核心功能是解释指令。
  2. 完成一条指令分三步：取指令、分析指令、执行指令。
  3. 取数指令流程：PC→MAR→M→MDR→IR，然后 OP(IR)→CU 分析，Ad(IR)→MAR→M→MDR→ACC 执行。
  4. 存数指令同理，只是执行阶段改为 ACC→MDR→M（按 Ad(IR) 给出的地址写入）。
  5. ax^2+bx+c 程序运行：程序经输入设备送入计算机，程序首地址→PC，启动后反复"取指—分析—执行"直至停机。
- **易错/考点**：PC 里放的是下一条指令地址，IR 里放的是指令本身；OP(IR) 送 CU、Ad(IR) 送 MAR 是两条不同通路。

1.3 计算机硬件的主要技术指标（18.6 分钟）

- **主旨**：给出机器字长、运算速度、存储容量三大指标的定义与常用度量。
- **关键点**：
  1. 机器字长指 CPU 一次能处理数据的位数，与 CPU 中寄存器的位数有关。
  2. 运算速度可用主频、核数与每核支持的线程数衡量。

3. 吉普森（Gibson）法：$T_M = \Sigma f_i \cdot t_i$，按各类指令在程序中出现的频度 f_i 与其执行时间 t_i 加权求平均。
4. CPI 表示执行一条指令所需的时钟周期数，MIPS 表示每秒执行百万条指令。
5. 存储容量指存放二进制信息的总位数，等于存储单元个数 × 存储字长；分主存容量与辅存容量。

- **易错/考点**：存储容量算的是"总位数"而非字节数，题目常要求换算成 MB/GB，注意 1B=8bit。

第 2 章 计算机的发展及应用

> 只有 4 讲，讲硬件代际更替与应用场景。信息密度低，可以先跳过，回头再补。

2.1 计算机的发展史-a1（17.4 分钟）

- **主旨**：从 ENIAC 出发，梳理计算机产生、代际划分与硬件技术演进。
- **关键点**：
  1. 世界上第一台电子计算机 ENIAC 于 1946 年在美国诞生，采用十进制运算。
  2. ENIAC 的基本指标：18 000 个电子管、1 500 个继电器、150 千瓦、30 吨、1 500 平方英尺、5 000 次加法/秒。
  3. 计算机产生的驱动力归根到底是需求（"需求、需求、还是需求"），加上技术发展的支撑。
  4. 五代划分按硬件器件：电子管（1946—1957，4 万次/秒）、晶体管（1958—1964，20 万次/秒）、中小规模集成电路（1965—1971，100 万次/秒）。
  5. 第四代大规模集成电路（1972—1977，1 000 万次/秒）、第五代超大规模集成电路（1978—现在，1 亿次/秒）。
- **易错/考点**：ENIAC 是十进制而非二进制，存储程序思想是后来冯·诺依曼结构才确立的。

2.1 计算机的发展史-a2（7.8 分钟）

- **主旨**：讲冯·诺依曼结构与 IAS 的逻辑组成，以及 IBM System/360 的里程碑意义。
- **关键点**：
  1. 冯·诺依曼系统结构的计算机把程序也当作数据存入存储器，实现存储程序控制。
  2. IAS 的逻辑结构由 Main Memory、Arithmetic Logic Unit、Program Control Unit、I/O Equipment 四部分互连组成。
  3. 运算器与控制器之间双向传递控制与状态，主存与 ALU、控制器之间传送数据与指令。
  4. IBM System/360 是系列机（同一体系结构、不同实现）的开端，确立了软件兼容的思想。
- **易错/考点**：IAS 框图是"以运算器为中心"的原始形态，与后来"以存储器为中心"的框图要区分。

2.1 计算机的发展史-b（8.3 分钟）

- **主旨**：讲微处理器的位宽演进、Intel 典型产品与摩尔定律。
- **关键点**：

1. 微处理器芯片自 1971 年出现，位宽沿 4 位（4004）→8 位→16 位→32 位→64 位演进。
2. Intel 8080 为 8 位（1974），8086 为 16 位（1979），80286 为 16 位（1982，13.4 万晶体管）。
3. 80386 32 位（1985，27.5 万）、80486 32 位（1989，120.0 万）、Pentium 64 位（准，1993，310.0 万）。
4. Pentium Pro（1995，550.0 万）、Pentium II（1997，750.0 万）、Pentium III（1999，950.0 万）、Pentium IV（2000，4 200.0 万）。
5. 摩尔（Moore）定律由 Intel 缔造者之一 Gordon Moore 提出：微芯片上集成的晶体管数目每三年翻两番。

- **易错/考点**：摩尔定律的表述是"每三年翻两番"，不是"每 18 个月翻一番"，答题按课件原文。

2.2 -2.3 计算机的应用和展望（12.1 分钟）

- **主旨**：列举计算机的六大应用领域，并展望智能计算与新器件方向。
- **关键点**：
  1. 科学计算和数据处理是计算机最基础的应用。
  2. 工业控制和实时控制要求响应及时，属于实时系统范畴。
  3. 网络技术应用包括电子商务、网络教育、敏捷制造。
  4. 其余应用为虚拟现实、办公自动化和管理信息系统、CAD/CAM/CIMS。
  5. 展望：让计算机具备类人智能要求速度足够快；芯片集成度受物理极限制约、制作成本按几何级数递增；替代硅芯片的方向有光计算机（用光子取代电子进行运算和存储）、DNA 生物计算机、量子计算机。
- **易错/考点**：CIMS 是计算机集成制造系统，与 CAD（设计）、CAM（制造）是三个不同层次的概念。

第 3 章 系统总线

> 总线是把五大部件粘起来的骨架。重点是三组性能指标（带宽、位宽、工作频率）和两种控制方式（仲裁、通信）。

3.1 总线的基本概念（13.3 分钟）

- **主旨**：说明为什么要用总线、总线的定义以及串行/并行两种传送方式。
- **关键点**：
  1. 总线的定义：连接各个部件的信息传输线，是各部件共享的传输介质。
  2. 采用总线可减少连线数量、便于部件扩充与系统集成，是部件间互连的标准方式。
  3. 总线上信息按位传送方式分串行（一位一位传）与并行（多位同时传）两类。
  4. 单总线结构：CPU、主存与各 I/O 接口都挂在一组系统总线上，结构简单但易成瓶颈。
  5. 双总线有两种：面向 CPU 的双总线（CPU 与主存之间加 M 总线、与 I/O 之间加 I/O 总线）、以存储器为中心的双总线（CPU 与主存间加存储总线）。
- **易错/考点**：面向 CPU 的双总线中 M 总线只连 CPU 与主存；以存储器为中心的双总线把存储总线独立出来，CPU 不再独占主存通路。

3.2 总线的分类（4.7 分钟）

- **主旨**：按连接范围把总线分为片内总线、系统总线、通信总线三类。
- **关键点**：
  1. 片内总线是芯片内部的总线，用于 CPU 芯片内各寄存器与 ALU 之间的连接。
  2. 系统总线是计算机各部件之间的信息传输线，按传送信息不同分为数据总线、地址总线、控制总线。
  3. 数据总线双向，其宽度与机器字长、存储字长有关。
  4. 地址总线单向，由 CPU 输出，位数与存储地址、I/O 地址空间有关。
  5. 通信总线用于计算机系统之间或计算机系统与其他系统（控制仪表、移动通信等）之间的通信。
- **易错/考点**：数据总线是双向的、地址总线是单向的，这是判断总线方向的固定考点。

3.3 总线特性及性能指标（14.4 分钟）

- **主旨**：给出总线的四大特性、七项性能指标，并用标准表对比典型总线。
- **关键点**：

1. 机械特性指尺寸、形状、管脚数及排列顺序。
2. 其余三大特性为电气特性、功能特性、时间特性。
3. 性能指标包括总线宽度、标准传输率、时钟同步/异步、总线复用、信号线数、总线控制方式及其他指标。
4. 标准对比：ISA 16 位/8 MHz/16 MBps，EISA 32 位/8 MHz/33 MBps，VESA 32 位/32 MHz/132 MBps。
5. PCI 32 位 33 MHz 为 132 MBps、64 位 66 MHz 为 528 MBps；AGP 32 位 66.7 MHz 为 266 MBps、133 MHz 为 533 MBps；USB 2.0 最高 480 Mbps。

- **易错/考点**：MBps 与 Mbps 差 8 倍；总线带宽 = 总线宽度/8 × 总线时钟频率，可用表中数据验算。

3.4 总线结构（10.7 分钟）

- **主旨**：从单总线推进到多总线，讲双/三/四总线与多层 PCI 的结构演进。
- **关键点**：

1. 双总线结构由主存总线与 I/O 总线组成，中间用通道（具有特殊功能的处理器）统一管理 I/O。
2. 三总线结构为主存总线 + I/O 总线 + DMA 总线，高速外设经 DMA 总线直接与主存交换数据。
3. 三总线的另一形式为局部总线（CPU 与 Cache）、系统总线、扩展总线三级。
4. 四总线结构在系统总线之外增设高速总线与扩展总线，把高速设备与低速设备分开挂接。
5. 传统微型机总线结构用 33 MHz、32 位数据通路的系统总线，再经标准总线控制器分出 8 MHz、16 位数据通路的 ISA/EISA 总线。

- **易错/考点**：多层 PCI 结构中桥（bridge）用于总线间转换，第一级桥连 CPU/存储器总线，逐级向下挂 PCI 设备。

3.5 总线控制-a（20.4 分钟）

- **主旨**：讲总线判优（仲裁）的基本概念与三种集中式判优方式。
- **关键点**：

1. 主设备（模块）指对总线有控制权的部件，只有它能发起总线传输。
2. 链式查询方式用三根控制线：BS（总线忙）、BR（总线请求）、BG（总线同意）。
3. 链式查询中 BG 信号沿 I/O 接口链逐级下传，越靠近总线控制部件的设备优先级越高。
4. 计数器定时查询方式增设一组设备地址线，由总线控制部件内的计数器给出计数值作为设备地址。
5. 计数器可改为每次从 0 开始（固定优先级）或从上次终点继续（循环优先级），优先级更灵活。

- **易错/考点**：链式查询线少但优先级固定、对电路故障敏感；计数器方式用地址线换取灵活性。

3.5 总线控制-b（15.2 分钟）

- **主旨**：讲独立请求方式与总线通信控制的目的、传输周期四阶段。
- **关键点**：
  1. 独立请求方式中每个设备各有独立的 BR_i 与 BG_i 线，由总线控制部件内部排队电路判优。
  2. 独立请求响应最快、优先级灵活，但控制线数量随设备数线性增长。
  3. 总线通信控制的目的是解决通信双方的协调配合问题。
  4. 总线传输周期分四阶段：申请分配阶段（主模块申请、总线仲裁决定）、寻址阶段（主模块向从模块给出地址和命令）、传数阶段、结束阶段。
  5. 总线通信方式分同步通信、异步通信、半同步通信、分离式通信四种。
- **易错/考点**：判优解决"谁用总线"，通信控制解决"用的过程中双方怎么配合"，两者层次不同。

3.5 总线控制-c（15.5 分钟）

- **主旨**：用波形图对比同步、半同步、分离式三种通信方式的时序差异。
- **关键点**：
  1. 同步通信由统一时标控制数据传送；输入时 T1 主模块发地址、T2 发读命令、T3 从模块提供数据、T4 撤销。
  2. 同步式数据输出把读命令换成写命令，数据由主模块在 T2 之后送上总线。
  3. 半同步通信是同步与异步的结合：发送方用系统时钟前沿发信号、接收方用后沿判断识别，并增设 WAIT（等待）响应信号。
  4. 半同步的 T_W 可插入一个或多个等待周期，使慢速从模块也能配合，T3 从模块提供数据、T4 撤销。
  5. 分离式通信把一个传输周期分为两个子周期：子周期 1 主模块申请占用总线、用完即放弃；子周期 2 从模块申请占用总线送数据，各模块均有权申请且不等对方回答。
- **易错/考点**：前三种方式的共同点是"从模块准备数据期间不占用总线"造成总线空闲，分离式正是为挖掘这段潜力而提出。

第 4 章 存储器

> 全课最长的一章（30 讲）。主线是「容量、速度、成本」三者不可兼得：用层次结构化解，用 Cache 局部性原理兑现，用芯片扩展把容量拼出来。

4.1 概述-4.1-a（20.3 分钟）

- **主旨**：按存储介质、存取方式、在计算机中的作用三种口径给存储器分类。
- **关键点**：
  1. 按存储介质分：半导体存储器（TTL、MOS，易失）、磁表面存储器（磁头、载磁体）、磁芯存储器（硬磁材料、环状元件）、光盘存储器（激光、磁光材料）。
  2. 按存取方式分：存取时间与物理地址无关的称随机访问，包括随机存储器 RAM（执行过程中可读可写）与只读存储器 ROM（只读）。
  3. 存取时间与物理地址有关的称串行访问，如磁带。
  4. 按在计算机中的作用分：主存储器、辅助存储器、高速缓冲存储器 Cache、Flash Memory。
  5. 主存又分 RAM（静态 RAM、动态 RAM）与 ROM（MROM、PROM、EPROM、EEPROM）。
- **易错/考点**：半导体存储器是易失的，磁表面/磁芯/光盘是非易失的；ROM 属主存但仍按随机方式访问。

4.1 概述-4.1-b（16.9 分钟）

- **主旨**：讲存储器三个特性的矛盾与缓存-主存、主存-辅存两级层次结构。
- **关键点**：
  1. 存储器三个主要特性是速度、容量、价格/位，三者相互制约。
  2. 层次自顶向下为寄存器、缓存、主存、磁盘、光盘、磁带：速度由快变慢、容量由小变大、价格/位由高变低。
  3. 寄存器与缓存属 CPU，主存属主机，磁盘/光盘/磁带属辅存。
  4. 缓存-主存层次解决速度矛盾，全部由硬件实现，对程序员透明。
  5. 主存-辅存层次解决容量矛盾，由硬件与操作系统共同实现，对应虚拟存储器、虚地址与逻辑地址。
- **易错/考点**：缓存-主存层次用"硬件"实现、主存-辅存层次靠"硬件+操作系统"，这是两级层次的分工差别。

4.2 主存储器（上）4.2-a1（14.6 分钟）

- **主旨**：给出主存的内部组成框图、与 CPU 的连线，以及大端/小端地址分配。

- **关键点**：

  1. 主存由存储体、读写电路、MDR、地址译码器与驱动器、MAR、控制电路组成。
  2. 地址总线经 MAR 送入译码器选中存储单元，数据总线经 MDR 与读写电路交换数据，读/写命令由控制电路发出。
  3. 主存中存储单元地址的分配有大端（高位字节地址为字地址）与小端（低位字节地址为字地址）两种方式。
  4. 以 12345678H 为例，大端存放顺序为 12 34 56 78，小端为 78 56 34 12。
  5. 技术指标：存储容量为主存存放二进制代码的总位数；存取时间为访问时间（读出时间、写入时间）；存取周期为连续两次独立存储器操作所需的最小间隔时间。

- **易错/考点**：大端是"高字节放低地址"、小端是"低字节放低地址"；存取周期含恢复时间，故大于存取时间。

4.2 主存储器（上）4.2-a2（12.7 分钟）

- **主旨**：讲半导体存储芯片的基本结构、两种译码驱动方式与片选线在容量扩展中的作用。
- **关键点**：

  1. 芯片内部结构为：地址线→译码驱动→存储矩阵→读写电路→数据线，另有片选线 CS/CE 与读/写控制线。
  2. 译码驱动方式有线选法与重合法两种。
  3. 线选法用一根地址线直接选中一行（或一组单元），连线简单但适合小容量。
  4. 重合法用行、列两组地址线交叉选中一个单元，可大幅减少选择线数量，适合大容量芯片。
  5. 用 16K×1 位的芯片组成 64K×8 位的存储器，需 4 组（字扩展）× 8 片（位扩展）共 32 片。

- **易错/考点**：总片数 = 字扩展倍数 × 位扩展倍数（16K→64K 为 4 倍、1 位→8 位为 8 倍，共 32 片）。

4.2 主存储器（上）4.2-a3（19.8 分钟）

- **主旨**：半导体存储芯片的两种译码驱动方式（线选法与重合法），以及六管静态 RAM 基本单元的电路构成与读操作。
- **关键点**：

  1. 线选法用地址译码器输出直接作字线：A3~A0 经地址译码器产生 0~15 共 16 条字线，位线 0~7 经读/写控制电路接 D0~D7，构成 16×8 矩阵。
  2. 重合法把地址分成 X、Y 两组分别译码，X 地址译码器产生 X0~X31、Y 地址译码器产生 Y0~Y31，行列交点选中 32×32 矩阵中的一个单元，可大幅减少选择线数目。
  3. 六管静态 RAM 单元由 T1~T4 组成的双稳态触发器保存 0/1，T5、T6 为行地址选择门控管，T7、T8 为列地址选择门控管。

4. 读操作时行选、列选同时有效，触发器状态经 T6、T8 送到位线 A 与 A′，再经读放（读出放大器）由 DOUT 输出。

- **易错/考点**：线选法选择线多、重合法选择线少但需行列两级译码，二者译码器输入的分工是常考的画图点。

4.2 主存储器（上）4.2-b（10.1 分钟）

- **主旨**：静态 RAM 基本电路的写操作过程，以及 Intel 2114 静态 RAM 芯片的外特性与内部 64×64 矩阵结构。
- **关键点**：

1. 写操作时行选使 T5、T6 开、列选使 T7、T8 开、写选择有效，DIN 分左右两路：左路 DIN 反相后经 T7—T5 送到 A 端，右路 DIN 经 T8—T6 送到 A′ 端，把触发器置成相应状态。
2. Intel 2114 外特性：A0~A9 共 10 根地址线、I/O1~I/O4 共 4 根数据线、WE 与 CS 低电平有效、另有 Vcc 与 GND，存储容量 1K×4 位。
3. 2114 内部为 64×64 的 RAM 矩阵，横向分第一~第四组，行地址译码 0~63、列地址译码 0~15，每组各经一套读写电路输出一位 I/O。
4. 重合法一次选中同一行中四个组各一个单元，故 4 位数据可同时读出或写入（画面标注为「选一次四列」）。

- **易错/考点**：2114 是 1K×4 位而非 1K×8 位，10 根地址线对应 1K 个单元、4 根数据线对应 4 位字长。

4.2 主存储器（上）4.2-c（16.6 分钟）

- **主旨**：动态 RAM 的三管、单管基本单元电路，以及 Intel 1103（三管）与 4116（单管）两种 DRAM 芯片的结构。
- **关键点**：

1. 三管动态 RAM 单元中 T1 为存储管（靠栅极电容 Cs 存电荷）、T2 为读选择管、T3 为写选择管，T4 由预充电信号控制给数据线预充电。
2. 三管单元的读出与所存信息相反、写入与输入信息相同（画面原文标注「读出与存信息相反，写入与输入信息相同」）。
3. Intel 1103 为 32×32 矩阵的三管动态 RAM，行地址 A9~A5 五根进行译码器、列地址 A4~A0 五根进列译码器，输出经读写控制电路到 D，并设刷新放大器。
4. 单管动态 RAM 4116 容量 16K×1 位，外部有 RAS、CAS、WE 三个控制输入，A6~A0 七位地址分两次打入行地址缓存器与列地址缓存器。
5. 4116 内部含存储单元阵列、基准单元、读出放大器、再生放大器、I/O 缓存器以及数据输入/输出寄存器。

- **易错/考点**：单管单元靠电容存电荷、需刷新且是破坏性读出，三管单元读出需先预充电，两者读出极性不要混。

4.2 主存储器（上）4.2-d1（8.6 分钟）

- **主旨**：4116（16K×1 位）单管动态 RAM 芯片的读、写工作原理。
- **关键点**：
  1. 4116 的 16K 单元组织成 128 条行线 × 128 列选择，每个单元为一只 MOS 管加电容 Cs，每条位线上接读出放大器。
  2. 读操作时被选中行的各单元信息送到各位线，经读出放大器放大，再由列选择（0~127）选出一位送入 I/O 缓冲。
  3. 数据经 I/O 缓冲后由数据输出端 DOUT 输出，读出后原存信息被破坏，需由读出放大器重新写入（再生）。
  4. 写操作时 DIN 经数据输入、I/O 缓冲、列选择写入被选中的单元。
- **易错/考点**：单管单元的读出是破坏性的，读出放大器同时承担再生（重写）作用，这是 DRAM 必须刷新的根源。

4.2 主存储器（上）4.2-d2（14.3 分钟）

- **主旨**：动态 RAM 的三种 refresh（刷新）方式——集中刷新、分散刷新、异步刷新，以及 DRAM 与 SRAM 的比较。
- **关键点**：
  1. 刷新按行进行、只与行地址有关，不需要列地址，因此刷新一行等于同时刷新该行上的所有单元。
  2. 集中刷新（存取周期 0.5 μs、128×128 矩阵）：前 3872 个周期共 1936 μs 用于读/写或维持，最后 128 个周期共 64 μs 逐行刷新，刷新时间间隔 2 ms。
  3. 分散刷新（存取周期 1 μs）：每个存取周期后紧跟一个刷新周期，刷新间隔为 128 个存取周期，把刷新分散到每个周期中，无长死区。
  4. 异步刷新（分散与集中结合，存取周期 0.5 μs）：每 15.6 μs 刷新一行，使每行在 2 ms 内被刷新一次。
  5. 最后以表格从存储原理等方面对比 DRAM 与 SRAM（表体内容画面未展开）。
- **易错/考点**：集中刷新存在 64 μs 的「死区」，分散刷新无死区但加长了存取周期，异步刷新是二者的折中。

4.2 主存储器（下）4.2-e（10.6 分钟）

- **主旨**：只读存储器 ROM 的演进与四类典型器件：掩模 ROM、PROM、EPROM、EEPROM 及 Flash Memory。
- **关键点**：
  1. ROM 的演进线索：早期由厂家写入 → 用户可自己写一次 → 可多次写但需能擦除 → 电可擦写但需特定设备 → 电可擦写并可直接连到计算机。
  2. 掩模 ROM（MROM）：行列选择线交叉处有 MOS 管表示存「1」，无 MOS 管表示存「0」，内容在制造时由掩模决定、不可改。
  3. PROM（一次性编程）：靠熔丝编程，熔丝烧断为「0」、熔丝未断为「1」，写入后不可再改。
  4. EPROM（多次性编程）：采用 N 型沟道浮动栅 MOS 管，D 端加正电压使电荷注入浮动栅，浮栅带电后 S 与 D 不导通表示「0」。
  5. EEPROM 为电可擦写，支持局部擦写与全部擦写；Flash Memory 为闪速型存储器，属电擦写类。
- **易错/考点**：MROM 的「有管为 1、无管为 0」与 PROM 的「熔丝断为 0、未断为 1」极性相反，最易记混。

4.2 主存储器（下）4.2-f-1（18.6 分钟）

- **主旨**：存储器容量扩展的三种方式——位扩展、字扩展、字位同时扩展的连线方法。
- **关键点**：
  1. 位扩展（增加存储字长）：用 2 片 2114（1K×4 位）组成 1K×8 位，10 根地址线两片共用，两片数据线分别接 D7~D4 与 D3~D0，合计 8 根数据线。
  2. 字扩展（增加存储字的数量）：用 2 片 1K×8 位组成 2K×8 位，需 11 根地址线，A9~A0 并联到各片，最高位 A10 经非门产生 CS0、CS1 两个片选。
  3. 字、位扩展：用 8 片 1K×4 位组成 4K×8 位，共需 12 根地址线、8 根数据线，即 4 组字扩展 × 每组 2 片位扩展。
  4. 扩展后各片的 WE 端并联接 CPU 的写命令，只有被片选选中的芯片才真正参与本次读写。
- **易错/考点**：地址线根数由「总容量」决定（4K 需 12 根），数据线根数由「字长」决定，片选由多余的高位地址译码产生。

4.2 主存储器（下）4.2-f-2（23.3 分钟）

- **主旨**：存储器与 CPU 连接的完整步骤，并用例 4.1 演示从写地址码到画连接图的全过程。
- **关键点**：

1. 连接步骤为：地址线的连接、数据线的连接、读/写命令线的连接、片选线的连接、合理选择存储芯片，最后还要考虑时序与负载。
2. 例 4.1 的 ROM 为 2K×8 位，占地址 0110 0000 0000 0000 ~ 0110 0111 1111 1111；RAM 为 1K×4 位，用 2 片位扩展成 1K×8，占 0110 1000 0000 0000 ~ 0110 1011 1111 1111。
3. 高 5 位 A15~A11 固定为 01100，A10~A0 作片内地址，因此 2K 的 ROM 用 A10~A0、1K 的 RAM 只用 A9~A0。
4. 74138 译码器的输出（画面标注 Y4、Y5）经门电路产生 ROM 与 RAM 的片选，MREQ 接 G2A、G2B 作译码使能，保证只在访存时才选中芯片。
5. CPU 侧 A15~A0、D7~D0、MREQ、WR 分别与存储芯片的地址线、数据线、片选、WE 相连。

- **易错/考点**：片选必须与 MREQ 联合控制，否则 CPU 访问 I/O 时也会误选中存储器。

4.2 主存储器（下）4.2-f-3（10.2 分钟）

- **主旨**：例 4.2 的完整解答——按「最小 4K 为系统程序区、相邻 8K 为用户程序区」的要求分配地址并画出连接图。
- **关键点**：

1. 解题三步：先写出对应的二进制地址码，再确定芯片的数量和类型，最后分配地址并画片选逻辑。
2. 系统程序区 4K 用 1 片 4K×8 位 ROM，地址为 0000 0000 0000 0000 ~ 0000 1111 1111 1111。
3. 用户程序区 8K 用 2 片 4K×8 位 RAM，分别为 0001 0000 0000 0000 ~ 0001 1111 1111 1111 与 0010 0000 0000 0000 ~ 0010 1111 1111 1111。
4. 连接图中 74138 的 Y0、Y1、Y2 分别选 ROM、RAM1、RAM2，A11~A0 作片内地址，MREQ 作译码使能。

- **易错/考点**：用户区 8K 必须用两片 4K 芯片而不是一片 8K，因为题目给出的芯片规格中只有 4K×8 位可用。

4.2 主存储器（下）4.2-g1（12.6 分钟）

- **主旨**：存储器校验的基本概念——合法代码集合与码距决定编码的检错、纠错能力。
- **关键点**：

1. 校验的前提是把写入的信息按某种规则编成「合法代码」，读出后检查是否仍落在合法代码集合内。
2. 编码的检测能力和纠错能力，与任意两组合法代码之间二进制位的最少差异数（码距）直接相关。
3. 码距为 1 时（如 {000, 001, 010, 011, 100, 101, 110, 111} 全码）检 0 位错、纠 0 位错。
4. 码距为 2 时（{000, 011, 101, 110}）检 1 位错、纠 0 位错；码距为 3 时（{000, 111}）检 1 位错、纠 1 位错。

5. 码距为 4 时（{0000, 1111}）检 2 位错、纠 1 位错；码距为 5 时（{00000, 11111}）检 2 位错、纠 2 位错。

- **易错/考点**：码距越大能力越强，但冗余位也越多；同一码距下「检错位数」总不少于「纠错位数」。

4.2 主存储器（下）4.2-g2（11.5 分钟）

- **主旨**：编码最小距离 L 与检错位数 D、纠错位数 C 的定量关系，以及汉明码的分组校验思想。
- **关键点**：

1. 编码的最小距离 L 定义为任意两组合法代码之间二进制位数的最少差异，检错纠错能力由 L 决定。
2. 定量关系为 L–1 = D + C（且 D ≥ C），L 为最小距离、D 为可检测错误的位数、C 为可纠正错误的位数，图中以 L=3 为例说明。
3. 汉明码采用奇偶检验并采用分组校验，其分组是一种「非划分方式」，同一代码位可同时属于多个校验组。
4. 7 位汉明码分成 3 组、每组配 1 位校验位，故 7 位中只有 4 位是数据位。
5. 分组用三个相交的圆表示：三圆公共区为第 7 位，两两相交区为第 3、5、6 位，各圆独占区为第 1、2、4 位。

- **易错/考点**：汉明码的分组是「非划分」的（位可以重叠分组），这正是它能定位出错位的原因。

4.2 主存储器（下）4.2-h（8.0 分钟）

- **主旨**：汉明码组成的三要素——检测位数、检测位位置、检测位取值。
- **关键点**：

1. 检测位的数目 k 由 2^k ≥ n + k + 1 确定（n 为数据位数），例如 n=4 时取 k=3。
2. 检测位放在第 1、2、4 位（即 2^i 的位置），其余位置依次放数据位。
3. 分组规则：第 i 个校验位负责二进制序号中第 i 位为 1 的所有位置，如第 1 组为形如 xxxxx1 的位置。
4. 各校验位的取值由所在组内其余各位按奇偶原则异或求得。
5. 三个校验位 P4P2P1 的组合直接给出出错位置：000 无差错、001 第 1 位、010 第 2 位、011 第 3 位、100 第 4 位、101 第 5 位、110 第 6 位、111 第 7 位。

- **易错/考点**：校验位必须放在 1、2、4、8... 位，才能让 P4P2P1 的二进制值恰好等于出错位号。

4.2 主存储器（下）4.2-i1（16.0 分钟）

- **主旨**：各检测位所承担检测小组的具体划分，并通过例 4.4、例 4.5 与练习完成汉明码的配置与纠错全过程。
- **关键点**：

1. C1 检测第 1、3、5、7、9、11… 位（二进制末位为 1），C2 检测第 2、3、6、7、10、11… 位（倒数第 2 位为 1），C4 检测第 4、5、6、7、12、13… 位，C8 检测第 8、9、10、11… 位。
2. 例 4.4：对 0101 按偶校验配置汉明码，n=4 由 2^k ≥ n+k+1 得 k=3，码位排列为 C1 C2 0 C4 1 0 1。
3. 校验位按组内异或求值：练习中 0011 得 C1 = 3⊕5⊕7 = 1、C2 = 3⊕6⊕7 = 0、C4 = 5⊕6⊕7 = 0，故其汉明码为 1000011。
4. 纠错时重新计算 P1 = 1⊕3⊕5⊕7、P2 = 2⊕3⊕6⊕7、P4 = 4⊕5⊕6⊕7，P4P2P1 的取值即为出错位号；练习 2 中 0101101 算得 P4P2P1 = 100，判定第 4 位错。
5. 例 4.5 给出收到 0100111 反求原信息；练习 3 表明按奇原则配置同一数据会得到不同码字（0011 的配奇汉明码为 0101011）。

- **易错/考点**：偶校验与奇校验只差校验位取反，算 P4P2P1 时不要漏掉最高位 P4，否则出错位号会判错。

4.2 主存储器（下）4.2-i2（12.5 分钟）

- **主旨**：提高访存速度的三条措施，重点讲单体多字系统与多体并行系统的高位交叉编址。
- **关键点**：

1. 提高访存速度的三条途径：采用高速器件、采用 Cache-主存层次结构、调整主存结构。
2. 单体多字系统在一个主存控制器下设一个存储体，一次读出 W 位送入单字长寄存器，再分送数据寄存器，可在器件速度不变时提高带宽。
3. 多体并行系统的高位交叉、顺序编址：M0 占 00 0000~00 1111、M1 占 01 0000~01 1111、M2 占 10 0000~10 1111、M3 占 11 0000~11 1111。
4. 高位交叉时地址的高位是体号、低位是体内地址，每个体的地址连续，故连续地址的访问集中在同一个体内。

- **易错/考点**：高位交叉与低位交叉的区分点是「体号在高位还是低位」，顺序编址属于高位交叉。

4.2 主存储器（下）4.2-i3（12.7 分钟）

- **主旨**：低位交叉（轮流）编址的地址分配、流水线存取条件，以及 SDRAM、RDRAM 两类高性能存储芯片。
- **关键点**：

1. 低位交叉、各个体轮流编址：M0 为 0000 00、0001 00…，M1 为 0000 01、0001 01…，即地址低位为体号、高位为体内地址。
2. 低位交叉的特点是在不改变存取周期的前提下增加存储器的带宽。
3. 设四体低位交叉、存取周期为 T、总线传输周期为 τ，为实现流水线方式存取应满足 T = 4τ。
4. 连续读取 4 个字所需时间为 T + (4−1)τ，时序图中 M0~M3 依次错开 τ 启动、重叠工作。

5. 高性能芯片：SDRAM（同步 DRAM）在系统时钟控制下进行读出和写入、CPU 无须等待；RDRAM 由 Rambus 开发，主要解决存储器带宽问题。

- **易错/考点**：T = 4τ 是四体流水不失步的条件，若 T < 4τ 则总线等待，若 T > 4τ 则体来不及准备。

4.2 课后习题 4.6 讲解视频（8.1 分钟）

- **主旨**：字长 32 位、容量 64KB 的存储器在按字编址与按字节编址两种方式下的寻址范围与地址分配。
- **关键点**：

1. 题目条件：某机字长 32 位，存储容量 64KB，问按字编址的寻址范围，以及按字节编址时主存地址与字节地址的分配。
2. 按字节编址时寻址范围就是 64K，即 65536 个字节单元。
3. 按字编址时寻址范围为 64K / (32/8) = 16K，即 16384 个字。
4. 按字节编址的地址分配：字地址 0、4、8...65532，每个字地址对应 4 个连续字节地址（0~3、4~7...），最后一个字占 65532~65535。

- **易错/考点**：容量单位是 B 而字长是位，必须先除以 8 换算，64KB÷4B = 16K 字。

4.2 课后习题 4.14 讲解视频（10.3 分钟）

- **主旨**：18 位地址、8 位字长的微型机用 4K×4 位芯片组成模块板结构存储器时的容量、片数与板选择计算。
- **关键点**：

1. 地址码 18 位，故最大主存空间为 2^18 × 8 位 = 256KB（板书写作 2^18×8b → 256KB）。
2. 每个模块板容量 32K×8 位，共需 256K/32K = 8 个模块板。
3. 每板用 4K×4 位芯片组成 32K×8 位，需 (32K/4K)×(8/4) = 8×2 = 16 片。
4. 全部存储器共需 16×8 = 128 片 RAM 芯片。
5. CPU 选择模块板：8 个模块板对应地址高 3 位，经译码产生 8 个板选信号，低 15 位作板内地址。

- **易错/考点**：片数要同时算字扩展倍数和位扩展倍数（8×2），漏乘位扩展会得到 8 片而非 16 片。

4.2 课后习题 4.15 讲解视频（23.7 分钟）

- **主旨**：给定芯片规格与地址区间要求，完成地址分配、芯片选型、片选逻辑与 CPU-存储器连接图的完整设计。
- **关键点**：

1. 条件：CPU 有 16 根地址线、8 根数据线，MREQ 低电平有效，R/W 高电平读、低电平写；可选 ROM（2K×8、4K×4、8K×8）与 RAM（1K×4、2K×8、4K×8）。

2. 要求 0~4K–1 为系统程序区（4K×8 位 ROM），4096~16383 为用户程序区（12K），两区合计 16K，故 A15A14 = 00。
3. 系统区用 2 片 2K×8 位 ROM：ROM1 占 0000 0000 0000 0000~0000 0111 1111 1111，ROM2 占 0000 1000 0000 0000~0000 1111 1111 1111。
4. 用户区用 3 片 4K×8 位 RAM：RAM1 为 0001 0000~0001 1111、RAM2 为 0010 0000~0010 1111、RAM3 为 0011 0000~0011 1111（均省写低位）。
5. 74138 对高位地址译码产生片选（画面标注 Y0~Y3），MREQ 接 G2A、G2B 作使能，A11~A0 作片内地址，8 根数据线 D7~D0 并联到各片。

- **易错/考点**：芯片数量由「容量/字长」双重匹配决定，本题 ROM 区必须用两片 2K×8 而不是一片 4K×4（字长不够）。

4.2 课后习题 4.16 讲解视频（14.2 分钟）

- **主旨**：8 片 8K×8 位 RAM 与 16 位地址 CPU 的连接、各片地址范围计算，以及片选故障与地址线断线的后果分析。
- **关键点**：

1. 条件：16 根地址线、8 根数据线，MREQ 低电平有效、R/W 控制读写，8 片 8K×8 位 RAM 经 74138 与 CPU 相连。
2. 8 片 8K×8 位正好 64K，故 A15~A13 三位送 74138 作变量输入产生 8 个片选，A12~A0 作片内地址。
3. 各片地址范围：RAM0 0000H~1FFFH、RAM1 2000H~3FFFH、RAM2 4000H~5FFFH、RAM3 6000H~7FFFH、RAM4 8000H~9FFFH、RAM5 A000H~BFFFH、RAM6 C000H~DFFFH、RAM7 E000H~FFFFH。
4. 故障分析：不论写哪片 RAM，以 A000H 起始的 RAM5 都出现相同数据，根本原因是 RAM5 的片选 CS 始终为低电平；可能为片选端与 R/W、与 MREQ 或与地线错连、短路。
5. 第 (4) 问讨论地址线 A13 与 CPU 断线并接高电平的后果，画面给出把 A13 改接高电平后的译码示意（该处标注不清晰）。

- **易错/考点**：片选恒低会使该片「永远被选中」，表现为与其他片数据重复，这是典型的片选错连故障。

4.3 课后习题 4.23 讲解视频（18.8 分钟）

- **主旨**：用 8 片 8K×8 位 RAM 构成 64KB 低位交叉编址的 8 体并行存储器，求连接图与各体地址范围。
- **关键点**：

1. 条件：16 根地址线、8 根数据线，M/IO 高电平访存、低电平访 I/O，WR 与 RD 均低电平有效。
2. 每个 RAM 体容量为 64KB/8 = 8KB，即 8K×8 位，8 个体并联后总容量恰为 64KB。

3. 低位交叉编址把地址最低 3 位 A2A1A0 作体号、A15~A3 作体内地址，故同一体内相邻单元地址相差 8。
4. 各体地址范围（十六进制）：RAM0 0000H~FFF8H、RAM1 0001H~FFF9H、RAM2 0002H~FFFAH、RAM3 0003H~FFFBH、RAM4 0004H~FFFCH、RAM5 0005H~FFFDH、RAM6 0006H~FFFEH、RAM7 0007H~FFFFH。
5. 74138 以 A2A1A0 为变量输入产生 8 个体选信号，M/IO 参与使能，各体数据线 D7~D0 并联。

- **易错/考点**：低位交叉的地址范围不是一个连续区间，而是步长为 8 的等差序列（末位恒为体号）。

4.3 高速缓冲存储器4.3-a1（17.9 分钟）

- **主旨**：Cache 的作用与工作原理——为什么用 Cache、主存与 Cache 的编址、命中与未命中、命中率。
- **关键点**：

1. 用 Cache 的目的是避免 CPU「空等」现象，解决 CPU 与主存（DRAM）之间的速度差异，形成 CPU—缓存（容量小、速度高）—主存（容量大、速度低）的层次。
2. 主存按块划分：n 位地址 = m 位主存块号 + b 位块内地址，共 M = 2^m 块，每块 B = 2^b 个字。
3. Cache 同样分块：地址 = c 位缓存块号 + b 位块内地址，共 C 块，并另设标记（tag）记录对应的是哪个主存块。
4. 命中指 CPU 欲访问的主存块已调入 Cache，可直接从 Cache 取数；未命中则需把该主存块从主存调入 Cache。
5. 命中率是 CPU 欲访问的信息在 Cache 中的比率，与 Cache 容量和块长有关，一般每块可取 4~8 个字；块长取一个存取周期内从主存调出的信息长度，如 CRAY_1 采用 16 体交叉、块长取 16 个存储字。

- **易错/考点**：Cache 与主存的块内地址位数 b 必须相同，否则块无法整块搬移，这是编址的关键约束。

4.3 高速缓冲存储器4.3-a2（11.1 分钟）

- **主旨**：Cache-主存系统的效率公式与 Cache 的基本结构框图。
- **关键点**：

1. 效率定义为 e =（访问 Cache 的时间 / 平均访问时间）× 100%，e 与命中率 h 直接相关。
2. 记命中率为 h、访问 Cache 的时间为 tc、访问主存的时间为 tm，平均访问时间由 h 对 tc 与 tm 加权得到。
3. 基本结构中 CPU 送出的主存地址被拆成主存块号与块内地址，主存块号送地址映像变换机构与 Cache 标记比较。
4. 命中则经 Cache 地址（块号 + 块内地址）从 Cache 存储体取出数据经数据总线送 CPU；未命中则启动 Cache 替换机构腾出空位。

5. 框图中有「直接通路」，使主存与 Cache 之间的整块数据传送不必绕经 CPU。

- **易错/考点**：效率公式的分子是「访问 Cache 的时间」而不是「平均访问时间」，e 恒小于 100%。

4.3 高速缓冲存储器4.3-a3（9.8 分钟）

- **主旨**：Cache 的读、写操作流程，重点解决写操作时 Cache 与主存的一致性问题。
- **关键点**：

1. 读流程：CPU 发出访问地址 → 判断是否命中 → 命中则访问 Cache 取出信息送 CPU → 结束。
2. 未命中时先判断 Cache 是否已满：满则执行替换算法腾出空位，未满则直接把新的主存块调入 Cache，同时访问主存取信息送 CPU。
3. 写直达法（Write-through）：写操作时数据既写入 Cache 又写入主存，写操作时间就是访问主存的时间。
4. 写直达法下 Cache 块退出时不需要对主存执行写操作，更新策略比较容易实现。
5. 写回法（Write-back）：写操作时只把数据写入 Cache 而不写入主存，仅当该 Cache 块被替换出去时才写回主存。

- **易错/考点**：写直达保证一致性但每次写都要访存；写回减少访存次数，但块被替换时必须写回，二者权衡是常考点。

4.3 高速缓冲存储器4.3-b1（11.5 分钟）

- **主旨**：Cache-主存地址映射的直接映射与全相联映射两种方式及其地址格式。
- **关键点**：

1. 直接映射的主存地址划分为主存字块标记 t 位、Cache 字块地址 c 位、块内地址 b 位，满足 m = t + c + b。
2. 直接映射中某一主存块只能固定映射到某一 Cache 行（块号对 2^c 取模），故只需一个 t 位比较器比较标记。
3. 每个 Cache 行设有效位（1 表示有效），只有标记相等且有效位为 1 才判为命中，否则不命中。
4. 全相联映射允许任一主存块放入任一 Cache 行，地址只分主存字块标记（m = t + c 位）与块内地址（b 位）两段。
5. 全相联的标记位数最多（要区分全部 2^m 个主存块），图中每个 Cache 块与主存各块之间都有映射连线，硬件比较最复杂。

- **易错/考点**：直接映射的标记不含 Cache 行号（行号由地址中的 c 位直接给出），全相联则把行号也并入标记。

4.3 高速缓冲存储器4.3-b2（14.1 分钟）

- **主旨**：组相联映射的地址划分与替换算法，并对三种映射方式作小结。
- **关键点**：
  1. 组相联映射把 Cache 分成 Q 组、每组 r+1 块（图中 r=1，即每组 2 块），组间采用直接映射、组内采用全相联映射。
  2. 主存地址划分为主存字块标记 s = t + r 位、组地址 q = c − r 位、块内地址 b 位，满足 m = s + q + b。
  3. 替换算法主要有先进先出（FIFO）算法与近期最少使用（LRU）算法。
  4. 小结：直接映射时某一主存块只能固定映射到某一缓存块；全相联映射时某一主存块能映射到任一缓存块；组相联映射时某一主存块只能映射到某一缓存组中的任一块。
- **易错/考点**：组相联的标记比直接映射多 r 位（因为组内 r+1 块都要靠标记区分），组地址比 Cache 块号少 r 位。

4.4 辅助存储器（20.5 分钟）

- **主旨**：讲辅助存储器（外存）为何不能与 CPU 直接交换信息，以及磁表面存储的读写原理、硬/软磁盘结构和光盘技术。
- **关键点**：
  1. 辅助存储器的根本特点是**不直接与 CPU 交换信息**，程序和数据必须先调入主存才能被使用；其技术指标为记录密度（道密度 `Dt`、位密度 `Db`）、存储容量、平均寻址时间、数据传输率、误码率。
  2. 存储容量 `C = n × k × s`（n 为记录面数、k 为每面磁道数、s 为每道存储字节数）；平均寻址时间 = 寻道时间 + 等待时间，其中等待时间取磁盘转半圈的时间。
  3. 数据传输率 `Dr = Db × V`，即位密度乘线速度；磁盘的存取速度取决于寻址时间与磁头读写时间两部分。
  4. 磁记录原理：写线圈通电流使磁层按方向磁化，写"1"/"0"对应两种相反的磁化状态；读时磁通变化在读线圈中感应出电势 `e`，读出"1"/"0"对应感应脉冲的正负极性。
  5. 硬磁盘分类：按磁头分固定磁头/移动磁头，按盘片分可换盘/固定盘；结构链为主机 → 磁盘控制器 → 磁盘驱动器 → 盘片，磁盘控制器是主机与驱动器之间的接口，负责命令转换、数据格式转换和读写控制。
  6. 硬盘与软盘对比：速度高/低、磁头浮动于盘面/接触盘片、盘片多为固定盘组不可换/可换盘片、价格高/低；光盘用激光读写，第一代采用非磁性介质不可擦写，第二代采用磁性介质可擦写。
- **易错/考点**：容量公式中的 n 是记录面数而不是盘片数（双面盘一片顶两面），平均等待时间取半圈而非整圈。

第 5 章 输入输出系统

> CPU 与外设速度差几个数量级，于是有四代 I/O 方式：查询→中断→DMA→通道。核心是把「谁在等谁」讲清楚。

5.1 概述 5.1-a1（16.5 分钟）

- **主旨**：第 5 章输入输出系统的总纲，交代本章四条主线并梳理 I/O 系统从"分散连接串行工作"到"总线连接并行工作"的发展。
- **关键点**：
  1. 本章四条主线：输入输出系统的发展概况、输入输出系统的组成、I/O 设备与主机的联系方式、I/O 设备与主机信息传送的控制方式。
  2. 早期阶段：设备**分散连接**，CPU 与 I/O 设备**串行工作**，采用程序查询方式，CPU 效率极低。
  3. 接口模块和 DMA 阶段：改为**总线连接**，CPU 与 I/O 设备**并行工作**，控制方式演进为中断方式和 DMA 方式。
  4. 输入输出系统由 **I/O 软件 + I/O 硬件**两大部分组成。
  5. I/O 指令是 CPU 指令的一部分，格式为 `操作码 | 命令码 | 设备码` 三段。
  6. 通道指令是通道自身的指令，需指出数组的首地址、传送字数、操作命令，如 IBM/370 的通道指令为 64 位。
- **易错/考点**：I/O 指令属于 CPU 指令集（由 CPU 执行），通道指令属于通道自己（由通道执行），二者层次不同不能混为一谈。

5.1 概述 5.1-a2（13.1 分钟）

- **主旨**：从编址、选址、传送、联络、连接五个维度说明 I/O 设备与主机"怎么连、怎么找、怎么送"。
- **关键点**：
  1. 编址方式两种：**统一编址**把 I/O 端口当主存单元，用取数/存数指令访问；**不统一编址**单独划分 I/O 地址空间，需专门的 I/O 指令。
  2. 设备选址靠**设备选择电路**识别本设备是否被选中（地址线经译码产生 SEL 信号）。
  3. 传送方式分**串行**与**并行**：并行一次传多位、线多速度快距离近，串行一位一位传、线少可远距离。
  4. 联络方式分**立即响应**与**异步应答**：异步工作采用应答信号，由 `Ready` 与 `Strobe` 一对信号完成握手，保证数据在双方都就绪时才传送。
  5. 连接方式之一为**辐射式连接**：每台设备都配一套控制线路和一组信号线，主机与各外设单独连线，设备多时线数爆炸。

- **易错/考点**：统一编址省掉了专用 I/O 指令但占用主存地址空间，不统一编址地址空间独立但指令集要增加 I/O 指令。

5.1 概述 5.1-b1（17.2 分钟）

- **主旨**：给出程序查询与程序中断两种控制方式的流程图，说明中断方式为何能做到 CPU 与 I/O 部分并行。
- **关键点**：

  1. 程序查询流程：CPU 向 I/O 发读指令 → CPU 读 I/O 状态 → 检查状态（未准备就绪则原地循环等待，出错转出错处理）→ 从 I/O 接口读一个字到 CPU → CPU 向主存写入一个字 → 未完成则回到读状态继续。
  2. 程序查询的本质缺陷是 CPU 必须反复读状态并等待，CPU 与 I/O 完全**串行**，无任何重叠。
  3. 程序中断方式把 I/O 工作拆成两段：**自身准备**（CPU 不查询，CPU 干自己的事）与**与主机交换信息**（CPU 暂停现行程序来传送）。
  4. 因此中断方式实现了 CPU 和 I/O 的**部分并行工作**：没有踏步等待现象，代价是要中断现行程序、增加现场保护开销。
  5. 程序中断流程：CPU 向 I/O 发读指令后转去做其他事情 → 设备工作 → 准备就绪 → 设备经 I/O→CPU 发中断请求 → CPU 响应，暂停现行程序转中断服务程序完成取数并写入主存。

- **易错/考点**：中断方式的并行是"准备阶段并行、传送阶段串行"，不是全程并行；这也是它优于查询、劣于 DMA 的原因。

5.1 概述 5.1-b2（13.7 分钟）

- **主旨**：介绍 DMA 方式的三大特征，并用时间轴把查询、中断、DMA 三种方式的 CPU 占用情况摆在一起比较。
- **关键点**：

  1. DMA 的三条特征：主存和 I/O 之间有一条**直接数据通道**；**不中断现行程序**；采用**周期挪用（周期窃取）**。
  2. 查询方式时间轴：CPU 执行现行程序 → 启动 I/O → CPU 查询等待并传输 I/O 数据（整段 CPU 被占用）→ CPU 执行现行程序，I/O 的准备及传送时间全被空耗。
  3. 中断方式时间轴：CPU 启动 I/O 后继续执行现行程序，只在**指令执行周期结束**处插入一小段"间"，转去执行中断服务程序完成传送。
  4. DMA 方式时间轴：CPU 启动 I/O 后继续执行，每到一个**存取周期结束**就挪用（窃取）一个存取周期完成主存与 I/O 之间的传送。
  5. 三种方式 CPU 工作效率排序为 DMA > 程序中断 > 程序查询，DMA 把 CPU 的损失压缩到单个存取周期量级。

- **易错/考点**：中断在"指令执行周期结束"响应，DMA 在"存取周期结束"挪用，两个时间粒度不同，这是区分二者的关键坐标。

5.2 IO设备（17.9 分钟）

- **主旨**：讲外部设备的构成与分类，以及键盘、显示器等典型输入输出设备的工作原理。
- **关键点**：

  1. 外部设备的构成链为：主机 → I/O 接口 → 设备控制器 → 机/电/磁/光部分，接口负责与主机打交道，控制器负责操纵设备本体。
  2. 按用途分三大类：**人机交互设备**（键盘、鼠标、打印机、显示器）、**计算机信息存储设备**（磁盘、光盘、磁带）、机—机通信设备。
  3. 键盘工作原理：检测按键 → 判断哪个键被按下 → 将此键翻译成 **ASCII 码**，这一套称为编码键盘法。
  4. 显示器的三种显示方式对应三种实现：字符显示用**字符发生器**（点阵字库），图形显示产生**主观图像**（几何描述），图像显示产生**客观图像**（逐点采样）。
  5. 其他设备：A/D、D/A 转换器完成模拟量与数字量的相互转换；**终端**由键盘和显示器组成，还要完成显示控制与存储、键盘管理及通信控制。
  6. 汉字处理包含汉字输入、汉字存储、汉字输出三个环节；多媒体技术单列一节讨论声音、图像等媒体。

- **易错/考点**：字符显示靠字符发生器（查字库出点阵），图形显示是主观图形、图像显示才是客观图像，三者不要混。

5.3 IO接口（18.9 分钟）

- **主旨**：回答"为什么非要设接口"，并把接口的功能、组成与分类一一对应起来。
- **关键点**：

  1. 设置接口的六个理由：实现设备的选择、实现数据缓冲达到速度匹配、实现数据串—并格式转换、实现电平转换、传送控制命令、反映设备状态（"忙""就绪""中断请求"）。
  2. 总线连接方式的接口电路上挂四类信号线：数据线、地址线（经**设备选择线**选中设备）、命令线、状态线。
  3. 功能与组成严格对应：选址功能 → 设备选择电路；传送命令 → 命令寄存器和命令译码器；传送数据 → 数据缓冲寄存器；反映设备状态 → 设备状态标记（完成触发器 `D`、工作触发器 `B`、中断请求触发器 `INTR`）。
  4. 接口的基本组成由数据缓冲寄存器 DBR、设备选择电路、设备状态标记、命令寄存器和命令译码器构成，由控制逻辑电路统一协调。
  5. 接口分类：按数据传送方式分并行接口（Intel 8255）与串行接口（Intel 8251）；按功能选择灵活性分可编程接口（8255、8251）与不可编程接口（Intel 8212）；按通用性分通用接口（8255、8251）

与专用接口（Intel 8279、Intel 8275）。

- **易错/考点**：8255 是并行、8251 是串行，8212 是不可编程，8279/8275 是专用，芯片型号最容易被张冠李戴。

5.4 程序查询方式（14.8 分钟）

- **主旨**：把程序查询的软件流程与接口电路的硬件握手（以输入为例）对上，看清一次查询传送的完整闭环。
- **关键点**：
  1. 单设备查询流程：检查状态标记 → 设备准备好？否就回到检查状态循环等待，是则交换数据。
  2. 完整程序流程：保存寄存器内容 → 设置计数值 → 设置主存缓冲区首址 → 启动外设 → 准备好？否则等待 → 传送一个数据 → 修改主存地址 → 未完成则继续下一轮。
  3. 接口电路（输入为例）的启动链：CPU 发启动命令 → 经命令译码 → 置工作触发器 `B = 1` 启动设备；设备工作结束时置完成触发器 `D = 1`（准备就绪）。
  4. 数据通路：输入数据先打入数据缓冲寄存器 DBR，再由 CPU 经数据线取走；`D` 的状态经状态线供 CPU 读取判断。
  5. 收尾：CPU 取走数据后，由设备选择信号 `SEL` 与读命令共同作用使 `D` 复位，接口回到初始状态等待下一次启动。
- **易错/考点**：`D` 不是设备自己清的，而是 CPU 取走数据后由 SEL 与读命令共同清除，否则下一次查询会误判"已就绪"。

5.5 程序中断方式5.5-a（15.6 分钟）

- **主旨**：建立中断的基本概念，以打印机为例说明 I/O 中断如何产生，并给出中断接口电路的四大功能部件。
- **关键点**：
  1. 中断的定义：CPU 暂停现行程序，转去执行中断服务程序，执行完后再返回原程序继续执行。
  2. 以打印机为例的产生过程：CPU 启动打印机后继续执行主程序 → 打印机准备 → 向 CPU 发中断请求 → CPU 响应中断，暂停现行程序转去传送数据 → 中断返回，继续执行主程序。
  3. 中断接口电路四大部件：中断请求触发器 `INTR` 与中断屏蔽触发器 `MASK`、排队器、中断向量地址形成部件、程序中断方式接口电路的基本组成。
  4. 两个触发器的电平语义：`INTR = 1` 表示**有中断请求**；`MASK = 1` 表示该设备**被屏蔽**（请求被封锁）。
  5. 排队器可用硬件实现（在 CPU 内或在接口电路中，典型为链式排队器），也可用软件排队（见第八章）。

- **易错/考点**：`INTR = 1` 是"有请求"、`MASK = 1` 是"被屏蔽"，两个 1 的含义正好相反，是选择题高频陷阱。

5.5 程序中断方式5.5-b（18.2 分钟）

- **主旨**：拆解链式排队器的优先级结构、中断向量地址如何形成入口地址，以及接口电路的整体连线。
- **关键点**：

  1. 链式排队器：`INTP1` ~ `INTP4` 逐级串接，设备 1°、2°、3°、4° 按优先级**降序**排列，前级 `INTP = 0` 时封锁后级的请求，天然形成固定优先级。
  2. 中断向量地址形成部件的两条路线：入口地址由**软件产生**（见第八章）或由**硬件向量法**产生。
  3. 硬件向量法的链条：由硬件产生**向量地址** → 再由向量地址找到**入口地址**，两步不能合并。
  4. 实例：排队器输出向量地址 `00010011`（13H），主存 12H/13H/14H 依次存放 `JMP 200`、`JMP 300`、`JMP 400`，从而跳到入口地址 200（打印机服务程序）、300（显示器服务程序）。
  5. 接口基本组成电路包含设备编码器、排队器、`INTR`、`MASK`、命令译码器、设备选择电路 SEL、DBR，并有至低一级和来自高一级排队器的级联线。

- **易错/考点**：向量地址 ≠ 入口地址——向量地址是"存放跳转指令的内存单元地址"，入口地址是服务程序第一条指令的地址，中间隔了一次 JMP。

5.5 程序中断方式5.5-c1（13.7 分钟）

- **主旨**：讲清 CPU 在什么条件下、什么时刻才响应中断，并把一次 I/O 中断从启动到返回的全过程串起来。
- **关键点**：

  1. 响应条件：允许中断触发器 `EINT = 1`；用**开中断**指令把 `EINT` 置 1，用**关中断**指令置 0（也可由硬件自动复位）。
  2. 响应时间：当 `D = 1`（完成触发器置位，时刻随机）且 `MASK = 0` 时，CPU 才在指令执行结束时响应。
  3. 一次输入中断的全过程：启动命令 → 设备工作 → 设备工作结束置 `D = 1` → 中断查询（`MASK = 0` 时经排队器）→ 向 CPU 发中断请求 → CPU 响应并发 `INTA` → 设备编码器给出向量地址 → 转中断服务程序。
  4. 中断服务程序四步：保护现场、中断服务、恢复现场、中断返回。
  5. 保护现场分两块：**程序断点保护**由中断隐指令（硬件）自动完成；**寄存器内容保护**由进栈指令（软件）完成；恢复现场用出栈指令，返回用中断返回指令。

- **易错/考点**：断点（PC）保护是硬件"中断隐指令"干的，通用寄存器保护才要程序员写进栈指令，二者责任主体不同。

5.5 程序中断方式5.5-c2（11.4 分钟）

- **主旨**：对比单重中断与多重中断的服务流程差别，并用时间轴展示主程序与服务程序如何抢占 CPU。
- **关键点**：
  1. **单重中断**：不允许中断现行的中断服务程序，服务全程保持关中断状态。
  2. **多重中断**：允许更高级别的中断请求打断正在执行的中断服务程序，形成中断嵌套。
  3. 单重中断流程：取指令 → 执行指令 → 有中断请求？→ 是则中断响应（程序断点进栈、关中断、向量地址→PC）→ 执行中断服务程序 → 返回，服务期间不再响应新请求。
  4. 多重中断流程与单重相同，差别只在进入服务程序后要**开中断**，使能更高优先级请求的嵌套。
  5. 主程序与服务程序抢占 CPU 的时间关系：CPU 启动外设后执行主程序 → I/O 做准备 → 准备就绪发中断请求 → CPU 响应转入服务程序传送数据 → 返回继续执行主程序，如此周而复始。
- **易错/考点**：单重与多重的分水岭是有没有在服务程序中"开中断"，而不是有没有排队器或优先级电路。

5.6 DMA方式5.6-a1（15.1 分钟）

- **主旨**：用两条数据通路的对比确立 DMA 的本质，并给出 DMA 与主存交换数据的三种方式。
- **关键点**：
  1. 两条通路的分野：中断方式下数据走"设备 → 中断接口 → CPU（输入指令进 ACC）→ 主存"；DMA 方式下数据走"设备 → DMA 接口 → 主存"，**不经过 CPU**。
  2. 三种交换方式：**停止 CPU 访问主存**、**周期挪用（周期窃取）**、**DMA 与 CPU 交替访问**。
  3. 停止 CPU 访问主存：控制最简单，但 CPU 处于不工作或保持状态，未充分发挥 CPU 对主存的利用率。
  4. 周期挪用：DMA 访存有三种可能——CPU 此时不访存、CPU 正在访存、CPU 与 DMA 同时请求访存；第三种情况下 CPU 把总线控制权让给 DMA，代价是 CPU 延迟一个存取周期。
  5. DMA 与 CPU 交替访问：把 CPU 工作周期分成 `C1`、`C2` 两段，`C1` 专供 DMA 访存、`C2` 专供 CPU 访存，不需总线请求/应答，效率最高但时序要求严。
- **易错/考点**：周期挪用下 CPU 只是被延迟一个存取周期而非停机，"停止 CPU 访问"才是真停，两者不能混。

5.6 DMA方式5.6-a2（10.9 分钟）

- **主旨**：讲 DMA 接口该干哪四件事，以及接口内部 AR、WC、DAR、BR 四个寄存器的分工。
- **关键点**：
  1. DMA 接口的四项功能：向 CPU 申请 DMA 传送、处理总线控制权的转交、管理系统总线并控制数据传送、确定数据传送的首地址与长度。

2. 内部寄存器：`AR` 主存地址寄存器、`WC` 字计数器、`DAR` 设备地址寄存器、`BR` 数据缓冲寄存器，外加 DMA 控制逻辑。
3. `AR` 与 `WC` 都带 `+1` 端：每传送一个字，主存地址加 1、字计数器减 1，`WC` 减到 0 即表示数据块传送结束。
4. 与设备侧的握手：设备发 `DREQ`（DMA 请求），DMA 接口回 `DACK`（DMA 响应），一请求一应答完成一次字传送。
5. 结构上 DMA 接口一侧挂系统总线（数据线、地址线），另一侧接设备，相当于设备与主存之间的"搬运工"。

- **易错/考点**：`AR` 存的是主存地址、`DAR` 存的是设备地址，两个"地址寄存器"职责完全不同，别按名字顺序想当然。

5.6 DMA方式5.6-b1（15.7 分钟）

- **主旨**：把 DMA 的完整工作过程拆成预处理、数据传送、后处理三段，并给出输入/输出两种方向的信号时序。
- **关键点**：

1. 三阶段：预处理 → 数据传送 → 后处理，前两者之间 CPU 与 DMA 是并行的。
2. 预处理（由 CPU 完成）：主存起始地址 → `AR`，设备地址 → `DAR`，传送数据个数 → `WC`，启动设备；随后 CPU 继续执行主程序，同时 DMA 完成一批数据传送。
3. 数据传送（由 DMA 控制逻辑循环完成）：DMA 请求 → 允许传送？→ 主存地址送总线、数据送 I/O 设备（或主存）、修改主存地址、修改字计数器 → 数据块传送结束？否则继续。
4. 输入方向时序：设备发 `DREQ` → 接口向 CPU 发 `HRQ` 申请总线 → CPU 回 `HLDA` 让出总线 → DMA 发地址与读写信号，数据经 `BR` 写入主存，同时 `AR+1`、`WC−1`。
5. 输出方向时序与输入相反：数据由主存读出，经 `BR` 送往设备，地址与计数同样逐字修改。
6. 后处理：校验送入主存的数是否正确、是否继续用 DMA、测试传送过程是否正确（错则转诊断程序），结束时向 CPU 申请一次程序中断。

- **易错/考点**：DMA 并非完全不用中断——数据块传完后仍要靠一次中断做后处理，"DMA 与中断无关"是错误说法。

5.6 DMA方式5.6-b2（15.9 分钟）

- **主旨**：比较 DMA 接口与系统的两种连接方式、DMA 与中断的差异，并说明选择型与多路型接口的工作原理。
- **关键点**：

1. 连接方式一：**具有公共请求线**的 DMA 请求——多个 DMA 接口共享一根 DMA 请求线和一根 DMA 响应线，靠优先级链或查询区分谁获准。
2. 连接方式二：**独立的 DMA 请求**——每个接口各有独立的请求/响应线，线数多但响应快、无冲突仲裁。
3. DMA 与中断的比较：数据传送上，中断靠**程序**（软件）、DMA 靠**硬件**；响应时间上，中断在指令执行周期结束时响应、DMA 在存取周期结束时响应。
4. 接口类型：**选择型**在物理上连接多个设备、在逻辑上只允许连接一个设备；**多路型**在物理上连接多个设备、在逻辑上允许连接多个设备同时工作。
5. 多路型工作原理：磁盘每 30 μs 发一次 DMA 请求、磁带每 45 μs 一次、打印机间隔很长；接口按优先级在 `T1`、`T2` 等时间片内分别服务各设备，实现多设备并行传送。

- **易错/考点**：选择型与多路型的区别全在"逻辑上"三个字——物理都连多个设备，逻辑上能否同时工作才是判据。

第 6 章 计算机的运算方法

> 计算量最大的一章（29 讲）。无符号数与有符号数的表示是入口，定点乘除的移位加法是主体，浮点的阶码尾数运算与 IEEE 754 是出口。

6.1 无符号数和有符号数6.1-a1（8.8 分钟）

- **主旨**：第 6 章的开篇导论，交代本章"数的表示—运算方法—运算器设计"三段式框架及其在第 3 篇 CPU 中的位置。
- **关键点**：
  1. 本章三大内容：计算机中数的表示、计算机的运算方法、运算器的设计。
  2. 本章五节：6.1 无符号数和有符号数、6.2 数的定点表示和浮点表示、6.3 定点运算、6.4 浮点四则运算、6.5 算术逻辑单元。
  3. 教材为唐朔飞《计算机组成原理》（第 2 版），普通高等教育"十二五"国家级规划教材、面向 21 世纪课程教材。
  4. 参考教材：Hennessy & Patterson《Computer Organization & Design: A Hardware/Software Interface》、Harris & Harris《Digital Design and Computer Architecture》。
  5. 课程内容组织上，CPU 由 ALU、CU、寄存器堆及内部总线构成，与存储器、I/O、系统总线共同组成计算机，本篇聚焦 CPU。
- **易错/考点**：6.1 的内部顺序是"无符号数 → 机器数与真值 → 原码 → 补码 → 反码 → 移码"，先建立机器数概念再谈各种码，顺序颠倒容易乱。

6.1 无符号数和有符号数6.1-a2（15.7 分钟）

- **主旨**：给出无符号数的表示范围，并建立"真值 vs 机器数"的区分与原码的定义式。
- **关键点**：
  1. 无符号数的表示范围完全由寄存器位数决定：8 位为 `0 ~ 255`（`2^8-1`），16 位为 `0 ~ 65535`（`2^16-1`），通式 `0 ~ 2^n-1`。
  2. **真值**是带符号的数，如 `+0.1011`、`-1100`，是人书写习惯的形式。
  3. **机器数**是符号数字化后的数，最高位用 0/1 表示正负：`+0.1011` → `0,1011`，`-1100` → `1,1100`。
  4. 原码（整数）定义：`[x]原 = 0, x`（`2^n > x ≥ 0`）；`[x]原 = 2^n - x`（`0 ≥ x > -2^n`），n 为整数的位数。
  5. 原码（小数）定义：`[x]原 = x`（`1 > x ≥ 0`）；`[x]原 = 1 - x`（`0 ≥ x > -1`）。

6. 书写规范：整数用逗号、小数用小数点把符号位与数值部分隔开，如 `0,1110` 与 `0.1110`。

- **易错/考点**：公式里的 n 是数值部分的位数（符号位不算），所以 8 位机整数 n = 7，模是 2^7 而不是 2^8。

6.1 无符号数和有符号数6.1-a3（15.1 分钟）

- **主旨**：用四个例题练熟"由原码反求真值"，并指出原码的 ±0 二义性与加减法不统一的致命缺陷。
- **关键点**：

1. 例 6.1：已知 `[x]原 = 1.0011`，由定义 `x = 1 − [x]原 = 1 − 1.0011 = −0.0011`。
2. 例 6.3：已知 `[x]原 = 0.1101`，符号位为 0 直接读出 `x = +0.1101`。
3. 例 6.4：`x = 0` 时 `[+0.0000]原 = 0.0000`、`[−0.0000]原 = 1.0000`；整数同理 `[+0]原 = 0,0000`、`[−0]原 = 1,0000`，故 `[+0]原 ≠ [−0]原`。
4. 原码的特点：简单、直观，符号位与数值部分彼此独立，转换不需运算。
5. 原码作加法的缺陷：同号相加可直接加；异号相加实际要做**减法**，且结果符号可正可负，运算规则不统一，硬件需同时配置加法器与减法器。
6. 改进思路由此产生：找一个与负数等价的正数来代替该负数，把"减"统一成"加"——这就是补码的出发点。

- **易错/考点**：原码的 0 有两种编码（±0），正是它比补码少表示一个数的根源，也是"原码不能直接判零"的原因。

6.1 无符号数和有符号数6.1-b1（18.2 分钟）

- **主旨**：借时钟与计数器的"模"直观引入补码，说明为何负数可以换成等价的加法，并给出整数补码定义式。
- **关键点**：

1. 时钟模型：指针在 6，要拨到 3，逆时针拨 3（`6−3=3`）与顺时针拨 9（`6+9=15`，超过 12 归零得 3）等效，故称 +9 是 -3 以 12 为模的补数。
2. 记作 `−3 ≡ +9 (mod 12)`；同理 `−4 ≡ +8 (mod 12)`、`−5 ≡ +7 (mod 12)`，减法由此变成加法。
3. 两条结论：一个负数加上"模"即得该负数的补数；一个正数与它的负数互为补数时，二者绝对值之和即为模数。
4. 计数器模型（模 16）：`1011 − 1011 = 0000`，而 `1011 + 0101 = 1,0000`，最高位的 1 自然丢弃，故 `−1011 ≡ +0101 (mod 2^4)`。
5. 整数补码定义：`[x]补 = 0, x`（`2^n > x ≥ 0`）；`[x]补 = 2^(n+1) + x`（`0 > x ≥ −2^n`，`mod 2^(n+1)`）。

6. 例：`x = +1010` → `[x]补 = 0,1010`；`x = −1011000` → `[x]补 = 2^(7+1) + (−1011000)`。

- **易错/考点**：补码的模是 `2^(n+1)`（含符号位共 n+1 位），原码公式用的是 `2^n`，模差一个 2 倍是最常见的记错点。

6.1 无符号数和有符号数6.1-b2（16.3 分钟）

- **主旨**：给出小数补码定义，推导"取反加 1"的快捷求法，并用三个例题练熟由补码反求真值。
- **关键点**：

1. 小数补码定义：`[x]补 = x`（`1 > x ≥ 0`）；`[x]补 = 2 + x`（`0 > x ≥ −1`，`mod 2`）。
2. 快捷求法的推导：设 `x = −1010`，则 `[x]补 = 2^(4+1) − 1010 = 100000 − 1010 = 1,0110`；而 `[x]原 = 1,1010`，两相比对即得"除符号位外每位取反、末位加 1"。
3. 口诀：真值为负时，补码 = 原码除符号位外每位取反、末位加 1；反之由补码求原码用同一套操作（取反加 1 是对合运算）。
4. 例 6.5：已知 `[x]补 = 0.0001`，由定义 `x = +0.0001`。
5. 例 6.6：已知 `[x]补 = 1.0001`，`x = [x]补 − 2 = 1.0001 − 10.0000 = −0.1111`。
6. 例 6.7：已知 `[x]补 = 1,1110`，`x = [x]补 − 2^(4+1) = 1,1110 − 100000 = −0010`，对应 `[x]原 = 1,0010`。

- **易错/考点**："取反加 1"只对数值位做，符号位保持 1；末位加 1 若向符号位进位，符号位仍为 1（如 `1,1110` 的补码场景）。

6.1 无符号数和有符号数6.1-b3（7.1 分钟）

- **主旨**：一组课堂练习，用整数、小数和 0 的边界值把补码/原码的换算彻底练一遍。
- **关键点**：

1. `x = +70 = 1000110`，`[x]补 = 0,1000110`、`[x]原 = 0,1000110`——正数三码完全相同。
2. `x = −70 = −1000110`，`[x]补 = 1,0111010`、`[x]原 = 1,1000110`，验证"取反加 1"。
3. `x = 0.1110` → `[x]补 = 0.1110`；`x = −0.1110` → `[x]补 = 1.0010`、`[x]原 = 1.1110`。
4. `x = 0.0000` 与 `x = −0.0000` 的补码都是 `0.0000`，即 `[+0]补 = [−0]补 = 0.0000`，补码的 0 唯一。
5. 边界值 `x = −1.0000`：由 `[x]补 = 2 + x = 10.0000 − 1.0000 = 1.0000`，而 `1.0000` 在原码中表示 -0，故 -1 没有原码表示。

- **易错/考点**：补码比原码/反码多表示一个负数（小数 -1、整数 -2^n），根源就是 0 只占一个编码，省下的编码给了最小负数。

6.1 无符号数和有符号数6.1-c（19.3 分钟）

- **主旨**：补上反码定义，把原码、补码、反码三种机器数做统一小结，并算出 8 位机各码的表示范围。
- **关键点**：
  1. 反码（整数）定义：`[x]反 = 0, x`（`2^n > x ≥ 0`）；`[x]反 = (2^(n+1) − 1) + x`（`0 ≥ x > −2^n`，`mod 2^(n+1) − 1`）。
  2. 反码（小数）定义：`[x]反 = x`（`1 > x ≥ 0`）；`[x]反 = (2 − 2^(−n)) + x`（`0 ≥ x > −1`），n 为小数的位数。
  3. 例 6.8：已知 `[x]反 = 0,1110`，得 `x = +1110`；例 6.9：已知 `[x]反 = 1,1110`，`x = [x]反 − (2^(4+1) − 1) = 1,1110 − 11111 = −0001`。
  4. 例 6.10：`[+0.0000]反 = 0.0000`、`[−0.0000]反 = 1.1111`，反码同样存在 ±0 二义性。
  5. 三码小结：最高位为符号位，整数用逗号、小数用小数点隔开；正数时原码 = 补码 = 反码；负数时符号位为 1，数值部分"除符号位外每位取反、末位加 1"得补码，"除符号位外每位取反"得反码。
  6. 8 位机（1 位符号位 + 7 位数值）整数范围：无符号 `0 ~ 255`；原码与反码均为 `−127 ~ +127`；补码为 `−128 ~ +127`。
- **易错/考点**：反码的 0 也是二义的（`0.0000` 与 `1.1111`），只有补码的 0 唯一，这正是补码能多表示一个负数的原因。

6.1 无符号数和有符号数6.1-d（15.7 分钟）

- **主旨**：针对"补码不便直观比较大小"的痛点引入移码，说明移码的定义、与补码的关系及其单调性。
- **关键点**：
  1. 补码的痛点：补码很难直接判断真值大小，如 `+21` 的补码 `0,10101`、`−21` 的补码 `1,01011`，若按无符号数比较会把 -21 判成更大。
  2. 移码定义：`[x]移 = 2^n + x`（`2^n > x ≥ −2^n`），x 为真值、n 为整数的位数。
  3. 例：`x = 10100` → `[x]移 = 2^5 + 10100 = 1,10100`；`x = −10100` → `[x]移 = 2^5 − 10100 = 0,01100`。
  4. 移码与补码的关系：只差符号位取反。如 `x = +1100100` 时 `[x]移 = 2^7 + 1100100 = 1,1100100`，而 `[x]补 = 0,1100100`；`x = −1100100` 时 `[x]移 = 0,0011100`。
  5. 对照表（n = 5）：真值 `−100000` 的移码为 `000000`（按无符号解释为 0），`+11111` 的移码为 `111111`（解释为 63），即移码按无符号数读出来恰好是"真值 + 32"。
  6. 移码特点：`[+0]移 = 2^5 + 0 = 1,00000`、`[−0]移 = 2^5 − 0 = 1,00000`，故 `[+0]移 = [−0]移`；n = 5 时最小真值 `−2^5 = −100000`，其移码为 `0,00000`。
- **易错/考点**：移码是"补码的符号位取反"（只反符号位，不反数值位），因此全 0 编码对应最小负数，全 1 编码对应最大正数。

6.2 数的定点表示和浮点表示（上）6.2-a（8.9 分钟）

- **主旨**：讲定点表示法——小数点位置按约定固定、不占存储位，分定点小数与定点整数两种格式并给出表示范围。
- **关键点**：
  1. 定点表示的核心：小数点按约定方式标出，物理上不存储小数点，位置由硬件约定固定不变。
  2. 定点小数格式：`Sf . S1S2…Sn`，`Sf` 为符号位，小数点固定在符号位与最高数值位之间，故 `|x| < 1`。
  3. 定点整数格式：`Sf S1S2…Sn .`，小数点固定在最低数值位之后，表示纯整数。
  4. 小数定点机的表示范围：原码与反码均为 `−(1 − 2^−n) ~ +(1 − 2^−n)`；补码为 `−1 ~ +(1 − 2^−n)`。
  5. 整数定点机的表示范围（板书给出原码一行）：`−(2^n − 1) ~ +(2^n − 1)`。
  6. 本章后续路线：定点表示 → 浮点表示（表示形式、表示范围、规格化形式、规格化）→ 举例 → IEEE 754 标准。
- **易错/考点**：定点小数补码能表示 -1（`1.000…0`）而原码不能，所以补码的小数范围是 `−1 ~ +(1−2^−n)`，两端不对称。

6.2 数的定点表示和浮点表示（上）6.2-b（18.9 分钟）

- **主旨**：说明为什么必须引入浮点表示，并给出浮点数的一般形式 N=S×r^j 与机器内的字段划分。
- **关键点**：
  1. 引入浮点数的两个动因：一是编程困难，程序员要自行调节小数点的位置；二是定点数表示范围小，为表示相差悬殊的数据需极长机器字长。
  2. 典型例证：太阳质量约 0.2×10^34 克，电子质量约 0.9×10^-27 克，两者相差 10^61 以上；若用定点数表示需 2^x>10^61，解得 x>203 位字长。
  3. 一般形式 `N = S × r^j`：S 为尾数（mantissa）、j 为阶码（exponent）、r 为尾数基值（radix），计算机中 r 取 2、4、8、16。
  4. 同一数据可写成多种浮点形式：11.0101 = 0.110101×2^10 = 1.10101×2^1 = 1101.01×2^-10 = 0.00110101×2^100，故必须约定规格化形式。
  5. 存储约定：S 取纯小数、可正可负；j 取整数、可正可负。
  6. 机器格式：阶码部分 j_f j_1…j_m（阶符 + 阶码数值）在前，尾数部分 S_f S_1…S_n（数符 + 尾数数值）在后，隐含小数点位于数符之后。
- **易错/考点**：阶符与数符各占 1 位，但"小数点位置"由格式隐含、不占位——小数点在数符与尾数最高位之间。

6.2 数的定点表示和浮点表示（下）6.2-c（13.1 分钟）

- **主旨**：由字长约束反推阶码/尾数位数分配，并给出规格化判据与左规、右规操作。
- **关键点**：
  1. 例题：机器数字长 24 位，欲表示 ±3 万的十进制数，因 2^14=16384、2^15=32768，取 15 位二进制即可覆盖 ±3 万，故尾数数值取 15 位，阶码数值 = 24-1-1-15 = 7 位。
  2. 规格化判据：r=2 时尾数最高位为 1；r=4 时尾数最高 2 位不全为 0；r=8 时最高 3 位不全为 0——基数不同，规格化形式不同。
  3. 左规：尾数左移 1 位、阶码减 1（r=2）；r=4 左移 2 位阶码减 2；r=8 左移 3 位阶码减 3。
  4. 右规：尾数右移 1 位、阶码加 1（r=2），r=4 右移 2 位阶码加 2，r=8 右移 3 位阶码加 3。
  5. m=4、n=10、r=2 规格化后的范围：最大正数 2^1111×0.1111111111 = 2^15×(1-2^-10)；最小正数 2^-1111×0.1000000000 = 2^-15×2^-1 = 2^-16。
- **易错/考点**：规格化后最小正数是 2^-16 而非 2^-15——尾数最小只能到 0.1（即 1/2），右端 9 个 0 使尾数恰为 0.5。

6.2 数的定点表示和浮点表示（下）6.2-d（15.9 分钟）

- **主旨**：用例 6.13、6.14 把真值写成定点数、浮点数及定点机/浮点机中的三种机器数，并定义机器零与 IEEE 754 标准。
- **关键点**：
  1. 例 6.13：x=+19/128，二进制 0.0010011；定点表示 0.0010011000；规格化浮点 0.1001100000×2^-10。
  2. 定点机中 [x]原=[x]补=[x]反=0.0010011000；浮点机中阶码取 5 位补码，[x]补 = 1,1110; 0.1001100000（阶码 -2）。
  3. 例 6.14：x=-58=-111010，定点 -0000 111010；浮点 -(0.1110100000)×2^110。
  4. 定点机三种机器数：[x]原=1,0000111010、[x]补=1,1111000110、[x]反=1,1111000101。
  5. 浮点机（阶码移码、尾数补码形式）：[x]原=0,0110;1.1110100000、[x]补=0,0110;1.0001100000、[x]反=0,0110;1.0001011111。
  6. 机器零两条规则：尾数为 0 时不论阶码为何值按机器零处理；阶码等于或小于它能表示的最小数时（如 m=4 即阶码 -16，编码 1,0000）不论尾数为何值也按机器零处理。
  7. IEEE 754：数符 S + 阶码（含阶符）+ 尾数，尾数规格化且非"0"有效位最高位"1"隐含；短实数 1+8+23=32 位，长实数 1+11+52=64 位。
- **易错/考点**：阶码用移码、尾数用补码时机器零为 0,0000; 0.00…0，全零便于"判 0"电路；阶码用补码时零的编码不唯一。

6.2 课后习题 6.16 视频讲解（33.1 分钟）

- **主旨**：用"数轴 + 边界点"的方法系统求出 16 位机器数在无符号、原码/补码定点与浮点格式下的表示范围。
- **关键点**：
  1. 题设：机器数字长 16 位、采用 1 位符号位，求 (1) 无符号数 (2) 原码定点小数 (3) 补码定点小数 (4) 补码定点整数 (5) 原码定点整数 的表示范围，答案用十进制。
  2. 无符号数：0 ~ 2^16-1（0 ~ 65535）。
  3. 原码定点小数：-(1-2^-15) ~ +(1-2^-15)；补码定点小数：-1 ~ +(1-2^-15)。
  4. 原码定点整数：-(2^15-1) ~ +(2^15-1)；补码定点整数：-2^15 ~ +(2^15-1)。
  5. 浮点格式：阶码 6 位（1 阶符 + 5 阶码）、尾数 10 位（1 数符 + 9 尾数）。
  6. 阶原尾原非规格化时，最大正数 0,11111; 0.111111111，正数范围为 2^31×(1-2^-9) ~ 2^-31×2^-9。
- **易错/考点**：补码比原码多表示一个负数（-1 或 -2^15），补码负数端不能写成 -(1-2^-15) 或 -(2^15-1)。

6.3 定点运算（上）6.3-a1（15.9 分钟）

- **主旨**：讲移位的数学意义与算术移位规则——符号位不变、空位按码制补"添补代码"。
- **关键点**：
  1. 移位本质是改变小数点位置：15,m = 1500,cm 相当于小数点右移 2 位。
  2. 机器中约定小数点不动，改为数据左移 2 位，故"左移"使绝对值扩大、"右移"使绝对值缩小。
  3. 算术移位针对有符号数，核心规则是符号位不变，空出的位按码制补添补代码。
  4. 添补代码表：正数（原码/补码/反码）补 0；负数原码补 0、负数反码补 1；负数补码左移补 0、右移补 1。
  5. 补码负数右移补 1 的由来：x=-0.x_1x_2...x_n 即 1.000...0，取补后 [x]补=1.$\bar{x}$_1$\bar{x}$_2...$\bar{x}$_n 1.00...0，最高位补的正是符号位 1。
- **易错/考点**：补码负数左移补 0、右移补 1，不能一律补 0；反码负数一律补 1，三种码制的填空规则必须分开记。

6.3 定点运算（上）6.3-a2（18.8 分钟）

- **主旨**：用 +26、-26 两个算例逐位验证三种机器数的移位结果，给出硬件实现并区分算术移位与逻辑移位。
- **关键点**：
  1. 例 6.16：A=+26=+11010，[A]原=[A]补=[A]反=0,0011010；因正数三种码制相同，左移/右移后三者结果一致。

2. 例 6.17：A=-26=-11010，原码 1,0011010=-26，左移一位 1,0110100=-52（符号位不动、数值位左移、低位补 0）。
3. 补码：1,1100110=-26；左移一位 1,1001100=-52；左移两位 1,0011000=-104；右移一位 1,1110011=-13；右移两位 1,1111001=-7。
4. 反码：1,1101001=-26；左移一位 1,1010011=-52；左移两位 1,0010111=-104（低位补 1）。
5. 硬件实现：移位寄存器左端/右端由门控选择添补代码，(a) 真值为正、(b) 负数原码补 0，(c) 负数补码左补 0 右补 1，(d) 负数反码补 1。
6. 逻辑移位针对无符号数：逻辑左移低位添 0、高位移丢；逻辑右移高位添 0、低位移丢。例 01010011 逻辑左移得 10100110，算术左移得 00100110（最高位 1 移去）。

- **易错/考点**：同一串 01010011 逻辑左移与算术左移结果不同——算术移位把最高位当符号位保留，逻辑移位把它当数据移丢。

6.3 定点运算（上）6.3-b2（14.8 分钟）

- **主旨**：补码加减运算的溢出判断（单符号位与双符号位）以及补码加减法的硬件配置。
- **关键点**：

1. 单符号位判溢出：参加操作的两个数（减法时为被减数与"求补"后的减数）符号相同，而结果符号与之不同，即为溢出。
2. 硬件判据：最高有效位的进位 ⊕ 符号位的进位 = 1，如 1⊕0=1、0⊕1=1 即判为溢出。
3. 双符号位（变形补码）：`[x]补 = x`（1>x≥0）或 `4+x`（0>x≥-1），运算按 mod 4 进行。
4. 判据：结果双符号位相同（00 或 11）未溢出，不同则溢出；10.×××× 为上溢，01.×××× 为下溢。
5. 硬件配置：n+1 位加法器、A 与 X 寄存器、求补控制逻辑（减法时对 X 求补）、溢出判断产生 V，以及 G_A、G_S 门控。

- **易错/考点**：双符号位判溢出看"两位是否相同"，且 10 表示上溢、01 表示下溢，与"1 为负"的直觉相反。

6.3 定点运算（上）6.3-c1（13.6 分钟）

- **主旨**：从笔算乘法归纳机器乘法的四条改造思路，推导原码一位乘的递推公式。
- **关键点**：

1. 笔算：A=-0.1101、B=0.1011，|A×B|=0.10001111，乘积符号由 x_0⊕y_0 心算单独求得。
2. 笔算四特点：符号位单独处理；乘数某一位决定是否加被乘数；4 个位积一起相加；乘积位数扩大一倍。
3. 改造思路：把"一次加 4 个位积"改为"边加边移位"，A·B = 0.1A+0.00A+0.001A+0.0001A 化为 0.1{A+0.1[0·A+0.1(A+0.1A)]}。

4. 递推式：`x*y* = 2^−1(y_1x* + 2^−1(y_2x* + … + 2^−1(y_nx* + 0)…))`，每步只做"加 x* 或不加"再右移一位。

- **易错/考点**：递推从乘数最低位 y_n 开始、部分积初值 z_0=0；每步都是"先加后移"，不是"先移后加"。

6.3 定点运算（上）6.3-c2（9.5 分钟）

- **主旨**：用竖式推演原码一位乘的完整过程，并小结其运算要点与硬件组成。
- **关键点**：

1. 竖式结构：左列部分积、中列乘数、右列说明；部分积初态 0.0000，乘数 1011，从最低位起逐位判定。
2. 乘数当前位为 1 则"加被乘数"，为 0 则"加 0"，随后整体右移一位形成新的部分积。
3. 乘数同步右移一位，末位移出丢弃，空出的高位正好用来存放部分积的低位，故部分积与乘数可拼在同一寄存器。
4. 小结：n=4 时加 4 次、移 4 次；被乘数只与部分积的高位相加。
5. 硬件：3 个寄存器（A 存部分积、Q 存乘数、X 存被乘数），其中 A、Q 两个具有移位功能，另需 n+1 位加法器与计数器 C。

- **易错/考点**：乘法由"加 + 移位"实现，加法次数等于尾数位数 n（本例 4 次），不是 2n 次。

6.3 定点运算（上）6.3-c3（17.1 分钟）

- **主旨**：给出原码一位乘的运算规则与例 6.21 的完整求解，并配硬件框图。
- **关键点**：

1. 规则：`[x·y]原 = (x_0⊕y_0).(0.x_1x_2…x_n)(0.y_1y_2…y_n) = (x_0⊕y_0).x*y*`，符号位异或、数值位取绝对值相乘。
2. 例 6.21：x=-0.1110、y=0.1101，部分积初值 0.0000，因 y 末位为 1 加 x*=0.1110 得 0.1110，逻辑右移得 0.0111。
3. 逐位判定 y 的 1、0、1、1，依次"加 x* / 加 0"后右移，得 xy = 0.10110110。
4. 结果：x_0⊕y_0 = 1⊕0 = 1，故 [x·y]原 = 1.10110110。
5. 特点为绝对值运算，符号与数值完全分离；硬件为 A、Q、X 三寄存器加 n+1 位加法器与"移位和加控制"。

- **易错/考点**：原码乘法中符号位不参与运算只做异或，部分积右移是逻辑右移（高位补 0），与负数原码的移位规则无关。

6.3 定点运算（上） 6.3-b1（18.5 分钟）

- **主旨**：补码加减运算公式（mod 2 或 mod 2^(n+1) 意义下的丢位）及例 6.18~6.20 的演算。
- **关键点**：
  1. 加法公式：整数 `[A]补+[B]补=[A+B]补 (mod 2^(n+1))`；小数 `[A]补+[B]补=[A+B]补 (mod 2)`。
  2. 减法公式：A-B = A+(-B)，故 `[A−B]补 = [A]补+[−B]补`，减法化为"加求补后的数"。
  3. 例 6.18：A=0.1011、B=-0.0101，[A]补=0.1011、[B]补=1.1011，相加得 10.0110，丢最高位后 0.0110，与直接验证 0.1011-0.0101=0.0110 一致。
  4. 例 6.19：A=-9、B=-5，[A]补=1,0111、[B]补=1,1011，相加得 11,0010，丢位后 1,0010 = -1110。
  5. 例 6.20：8 位机 A=15、B=24 求 A-B，[A]补=0,0001111、[B]补=0,0011000、[-B]补=1,1101000，相加得 1,1110111=[A-B]补，真值 -1001 = -9。
- **易错/考点**：符号位产生的进位必须丢掉（mod 运算），但该进位不能当溢出依据——溢出要靠符号位比较或双符号位判断。

6.3 定点运算（下）6.3-d2（13.2 分钟）

- **主旨**：原码除法的恢复余数法：规则说明与例 6.24 的完整推演。
- **关键点**：
  1. 原码除法约定：`[x/y]原 = (x_0⊕y_0).(x*/y*)`，商符由异或求得，数值部分为绝对值相除。
  2. 恢复余数法规则：余数 R_i>0 上商"1"，下一步做 2R_i - y；*余数 R_i<0 上商"0"，先 +y* 恢复余数再左移。
  3. 例 6.24：x=-0.1011、y=-0.1101，[x]原=1.1011、[y]原=1.1101、[y]*补=0.1101*、*[-y]*补=1.0011，商符 1⊕1=0。
  4. 首步 0.1011+1.0011=1.1110 为负，上商 0；加 0.1101 恢复为 0.1011，再左移得 1.0110 继续。
  5. 迭代完成得 x/y = 0.1101，故 [x/y]原 = 0.1101，余数为正说明末位上商 1。
- **易错/考点**：恢复余数法每步可能多做一次加法，加法次数随余数符号浮动、步数不固定，这正是引入不恢复余数法的动机。

6.3 定点运算（下）6.3-d3（14.6 分钟）

- **主旨**：由恢复余数法推导不恢复余数法（加减交替法），完成例 6.25 并给出硬件配置。
- **关键点**：
  1. 推导关键：若 R_i<0，恢复余数后左移再减 y，*即 2(R_i+y)*-y* = 2R_i+y*，于是"恢复"这一步可以省掉。

2. 加减交替规则：上商"1"后做 2R_i - y；*上商"0"后做 2R_i + y*，加减交替、不再单独恢复。
3. 例 6.25 同用 x=-0.1011、y=-0.1101：1.1110（商 0）→左移 1.1100+0.1101=0.1001（商 1）→左移 1.0010+1.0011=0.0101（商 1）→左移 0.1010+1.0011=1.1101（商 0）。
4. 结果 x/*y* = 0.1101，[x/y]原 = 0.1101，与恢复余数法完全一致。
5. 特点：上商 n+1 次（第一次上商判溢出），移 n 次，加 n+1 次，步数固定。
6. 硬件：n+1 位可加可减加法器、A（余数）、Q（商）、X（除数）寄存器、移位和加减控制逻辑、计数器 C 及 C_n、S、V。

- **易错/考点**：末步若余数为负须再加一次 y* 才能得到真余数；商符仍由 x_0⊕y_0 单独决定，不参与迭代。

6.3 定点运算（下） 6.3-d1（14.0 分钟）

- **主旨**：用笔算除法分析机器除法的难点，逐条对照给出机器化改造方案。
- **关键点**：

1. 笔算：x=-0.1011、y=0.1101，得 x÷y = -0.1101、余数 0.00001111，商符由心算确定。
2. 笔算四特点：商符单独处理；上商靠心算比较大小；余数不动、低位补"0"；每次减"右移一位的除数"；需 2 倍字长加法器。
3. 机器化改造：商符由符号位异或形成；用 |x|-|y|>0 上商 1、|x|-|y|<0 上商 0 代替心算。
4. 余数改为"左移一位、低位补 0"而除数不动，效果等价于笔算中的除数右移。
5. 用 1 倍字长加法器配双倍字长寄存器，代替笔算所需的 2 倍字长加法器。

- **易错/考点**：机器除法是"余数左移、除数不动"，与笔算"余数不动、除数右移"方向相反，两者等价，写过程时不可混用。

6.4 浮点四则运算（下）6.4-b（11.8 分钟）

- **主旨**：规格化的定义与判据，以及左规、右规两种规格化操作。
- **关键点**：

1. 规格化定义：r=2 时要求 1/2 ≤ |S| < 1，即尾数绝对值落在 [0.5, 1)。
2. 判据：S>0 时真值 0.1×…×，原码/补码/反码均为 0.1×…×；S<0 时真值 -0.1×…×，原码 1.1×…×，补码与反码为 1.0×…×。
3. 特例 S=-1/2=-0.100…0：[S]原=1.100…0，[S]补=1.100…0，即补码以 1.0 开头仍算规格化。
4. 左规：尾数左移一位、阶码减 1，直到数符与第一数位不同为止；上例 [x+y]补=00,11; 11.1001 左规为 00,10; 11.0010，即 x+y=(-0.1110)×2^10。
5. 右规：尾数右移一位、阶码加 1，用于尾数求和后出现 01.××× 或 10.××× 的双符号位溢出情形。

- **易错/考点**：补码规格化看"数符与第一数位是否相同"，不是看最高位是否为 1；1.0 开头（真值 -0.5）属于规格化，不能用原码判据套补码。

6.4 浮点四则运算（下）6.4-c（18.8 分钟）

- **主旨**：浮点加减的完整五步流程（对阶→尾数求和→规格化→舍入→溢出判断）与例 6.27、6.28 的演算。
- **关键点**：
  1. 例 6.27：x=0.1101×2^10、y=0.1011×2^01，[x]补=00,010; 00.110100，[y]补=00,001; 00.101100，阶差 [Δj]补=00,010-00,001=00,001。
  2. 对阶后尾数求和得 01.001010，双符号位为 01 需右规：右规为 00,011; 00.100101，即 x+y=0.100101×2^11。
  3. 例 6.28：x=(-5/8)×2^5、y=(7/8)×2^4，尾数 [S_x]补=11.101100 加 [-S_y]补=11.001000 得 110.110100，右规为 11,101; 11.011010。
  4. 结果 x-y=(-0.100110)×2^11 = (-19/32)×2^3。
  5. 舍入两法：一是"0 舍 1 入"法（多余位最高位为 1 则末位加 1），二是"恒置 1"法（直接令末位恒为 1）；对阶右移与右规都可能丢位，必须舍入。
- **易错/考点**：对阶一律"小阶向大阶看齐"（小阶数的尾数右移），若反过来让大阶向小阶看齐会丢失高位有效数字。

6.4 浮点四则运算（下） 6.4-a（14.6 分钟）

- **主旨**：浮点加减运算的总体框架与对阶原则——求阶差、小阶向大阶看齐。
- **关键点**：
  1. 浮点加减五步：对阶 → 尾数求和 → 规格化 → 舍入 → 溢出判断，另配举例。
  2. 设 `x = S_x·2^jx`、`y = S_y·2^jy`，阶差 `Δj = j_x − j_y`，按 =0、>0、<0 三种情形分别处理。
  3. 对阶原则：小阶向大阶看齐，即把小阶数的尾数右移 |Δj| 位、阶码加 |Δj|，使其阶码与大阶数一致。
  4. 例：x=0.1101×2^01、y=(-0.1010)×2^11，[Δj]补=[j_x]补-[j_y]补=00,01+11,01=11,10，阶差为 -2。
  5. 因阶差为负，将 x 的尾数右移 2 位、阶码加 2，得 [x]补=00,11; 00.0011，与 [y]补=00,11; 11.0110 阶码一致后再求和。
- **易错/考点**：阶差为负说明 x 的阶小，应移 x 而不是移 y；右移必然丢失尾数低位，这是浮点加法误差的主要来源。

6.5 算术逻辑单元 6.5-a（16.7 分钟）

- **主旨**：ALU 的电路模型与并行加法器的进位逻辑，引出快速进位链的必要性。

- **关键点**：

  1. 6.5 提纲：一、ALU 电路；二、快速进位链（1. 并行加法器 2. 串行进位链 3. 并行进位链：单重分组、双重分组跳跃进位链）。
  2. ALU 是组合逻辑电路：输入 A_i、B_i，控制端 K_i 取不同值则输出 F_i 不同；四位 ALU 74181 中 M=0 作算术运算、M=1 作逻辑运算，S_3~S_0 选择具体运算。
  3. 并行加法器由 n 个全加器 FA 串成，第 i 位和 `S_i = Ā_iB_iC_{i-1} + Ā_iB_iC_{i-1} + A_iB̄_iC_{i-1} + A_iB_iC_{i-1}`。
  4. 进位 `C_i = A_iB_i + (A_i+B_i)C_{i-1}`，记 d_i=A_iB_i（本地进位）、t_i=A_i+B_i（传送条件），则 `C_i = d_i + t_iC_{i-1}`。
  5. 串行进位链：C_0=d_0+t_0C_{-1}、C_1=d_1+t_1C_0、C_2=d_2+t_2C_1、C_3=d_3+t_3C_2，进位逐级串行传送。

- **易错/考点**：加法器的延迟瓶颈在进位链而非求和电路，提高速度的关键是减少进位传播的级数。

6.5 算术逻辑单元 6.5-b（11.1 分钟）

- **主旨**：并行进位链（先行进位）的展开公式与单重分组跳跃进位链的结构。
- **关键点**：

  1. 并行进位思想：n 位加法器的进位同时产生，把 C_i 展开为只含 d、t、C_{-1} 的与或式。
  2. 四位展开：C_0=d_0+t_0C_{-1}；C_1=d_1+t_1d_0+t_1t_0C_{-1}；
     C_2=d_2+t_2d_1+t_2t_1d_0+t_2t_1t_0C_{-1}；
     C_3=d_3+t_3d_2+t_3t_2d_1+t_3t_2t_1d_0+t_3t_2t_1t_0C_{-1}。
  3. 单重分组跳跃进位链：n 位全加器分若干小组，组内进位同时产生，组与组之间采用串行进位。
  4. n=16 分 4 组：第 1 组产生 C_12~C_15，第 2 组 C_8~C_11，第 3 组 C_4~C_7，第 4 组 C_0~C_3。

- **易错/考点**：并行进位以"与门/或门级数增加"换取"进位级数减少"；单重分组仍是组内并行、组间串行，这是它速度受限的根源。

6.5 算术逻辑单元 6.5-c（14.6 分钟）

- **主旨**：双重分组跳跃进位链——大组本地进位 D、传送条件 T 的构造与延迟对比。
- **关键点**：

  1. 大组进位展开：`C_3 = d_3+t_3d_2+t_3t_2d_1+t_3t_2t_1d_0+t_3t_2t_1t_0C_{-1} = D_8 + T_8C_{-1}`。
  2. D_8 为第 8 小组的本地进位，与外来进位无关；T_8 为小组传送条件，也与外来进位无关，只负责传递外来进位。

3. 同理 C_7=D_7+T_7C_3、C_11=D_6+T_6C_7、C_15=D_5+T_5C_11，进一步展开得 C_7 = D_7+T_7D_8+T_7T_8C_{-1}。
4. 小组进位线路只需产生低 3 位进位和本组的 D、T；大组进位由"第二重进位链"统一产生。
5. 延迟对比（n=16）：d_i、t_i、C_{-1} 形成后，经 2.5t_y 产生 C_0~C_2 及 D_5~D_8、T_5~T_8，经 5t_y 产生 C_3、C_7、C_11、C_15，经 7.5t_y 产生全部进位；单重分组需 10t_y，纯串行需 32t_y。

- **易错/考点**：D 与 T 都"与外来进位无关"——D 是本组自产的进位，T 是本组的传递系数，两者含义不可互换。

第 7 章 指令系统

> 指令系统是软硬件的分界面。格式决定能编出多少种指令，寻址方式决定操作数从哪来，RISC 与 CISC 是两条设计哲学。

7.1 机器指令 7.1-a（12.2 分钟）

- **主旨**：说明指令系统在软硬件交界处的地位，给出指令的一般格式与操作码的定长/变长方案。
- **关键点**：
  1. 第 7 章定位：指令系统是软件与硬件的交界面，向上支撑程序语义、向下规定硬件必须实现的操作。
  2. 指令一般格式 = 操作码字段 + 地址码字段；操作码反映"机器做什么操作"，地址码给出操作数与结果的位置。
  3. 指令格式三要素：操作码（opcode）、地址码、寻址方式（addressing mode）。
  4. 操作码长度固定：适用于指令字长较长的情况，译码简单，典型为 RISC 及 IBM 370。
  5. 操作码长度可变：须配合扩展操作码技术，在地址码位数减少时相应增加操作码位数。
- **易错/考点**：操作码定长不等于指令字长固定；RISC 是"定长操作码 + 定长指令字"，两者常被合并考查。

7.1 机器指令 7.1-b（10.5 分钟）

- **主旨**：扩展操作码技术——用地址码位数的减少换取操作码位数的增加。
- **关键点**：
  1. 原理：操作码位数随地址码个数的减少而增加，用短操作码表示常用指令、长操作码表示少用指令。
  2. 4 位操作码：0000~1110 共 15 种编码作三地址指令，保留 1111 作为扩展前缀。
  3. 扩展：1111 0000~1111 1110 共 15 种作二地址指令，再保留 1111 1111 继续扩展。
  4. 依此得到 15 条一地址指令（12 位操作码）与 16 位操作码的零地址指令。
  5. 数量关系：三地址指令每减少一种，最多可多构成 2^4 = 16 种二地址指令。
- **易错/考点**：每一级必须"留一个编码"给下一级，若把 1111 用尽就无法继续扩展；故每级最多 15 条而非 16 条。

7.1 机器指令 7.1-c（17.4 分钟）

- **主旨**：四/三/二/一/零地址格式的访存次数与寻址范围，以及指令字长的决定因素。
- **关键点**：

1. 四地址：OP A1 A2 A3 A4，`(A1)OP(A2)→A3`，A4 为下一条指令地址；32 位指令按 8+6+6+6+6 分配时寻址范围 2^6=64，访存 4 次。
2. 三地址：OP A1 A2 A3，`(A1)OP(A2)→A3`；用 PC 代替 A4 即得三地址格式。
3. 二地址：OP A1 A2，`(A1)OP(A2)→A1` 或 `→A2`；8+12+12 分配时寻址范围 2^12=4K；若结果存于 ACC 则访存 3 次。
4. 一地址：OP A1，`(ACC)OP(A1)→ACC`，隐含使用累加器 ACC。
5. 指令字长取决于操作码长度、操作数地址长度、操作数地址个数；分指令字长固定（指令字长 = 存储字长）与可变两种。
6. 小结：用寄存器等硬件资源代替地址码字段，可扩大寻址范围、缩短指令字长、减少访存次数；地址字段为寄存器时写作 OP R1,R2,R3 / OP R1,R2 / OP R1。

- **易错/考点**：二地址指令结果覆盖源操作数会破坏 A1 或 A2，需保留原值时必须改用三地址或先做保存。

7.2 操作数类型和操作类型 7.1-a（12.6 分钟）

- **主旨**：操作数的四种类型与数据在存储器中的三种存放方式及其取舍。
- **关键点**：

1. 操作数类型：地址（无符号整数）、数字（定点数、浮点数、十进制数）、字符（ASCII）、逻辑数（用于逻辑运算）。
2. 存放方式 a：从任意位置开始存储——不浪费存储资源，但一个数据可能跨两个存储字，需两次访存。
3. 存放方式 b：从一个存储字的起始位置开始访问——任何类型数据都在一个存取周期内完成，读写控制简单，但浪费存储资源。
4. 存放方式 c：边界对准方式，从地址的整数倍位置开始访问，兼顾访存次数与空间利用率，仍有一定浪费。
5. 讨论以存储字长 64 位、机器字长 32 位为例，在字节编址下数据的起点决定其是否跨字。

- **易错/考点**：三种方式的取舍本质是"空间利用率 vs 访存次数"，边界对准是二者的折中，不是最优解。

7.2 操作数类型和操作类型 7.2-b（9.2 分钟）

- **主旨**：五类操作类型——数据传送、算术逻辑、移位、转移、调用返回与陷阱。
- **关键点**：

1. 数据传送按源/目的组合分四种：寄存器→寄存器（MOVE）、存储器→寄存器（LOAD）、寄存器→存储器（STORE）、存储器→存储器（MOVE）；另有置"1"、清"0"（MOVE、PUSH、POP）。
2. 算术逻辑运算：加、减、乘、除、与、或、非、异或等，由 ALU 完成。
3. 移位操作：算术移位、逻辑移位、循环移位（分带进位与不带进位两种）。

4. 转移：无条件转移 JMP；条件转移按标志位判——结果为零（Z=1）用 JZ、结果溢出（O=1）用 JO、结果有进位（C=1）用 JC，另有跳过一条指令 SKP。
5. 调用与返回：CALL 保存返回地址并转入子程序，RETURN 取回返回地址；支持嵌套，如主程序 2000 处 CALL SUB1，SUB1 在 2400 处又 CALL SUB2。
6. 陷阱（Trap）：属意外事故引起的中断，一般不提供用户直接使用，出现事故时由 CPU 自动产生并执行（隐指令）；也可设用户可用的陷阱指令，如 8086 的 `INT TYPE` 软中断，用于完成系统调用。

- **易错/考点**：陷阱具有"指令 + 中断"双重身份——软中断由指令主动触发、硬中断由外部事件触发，但两者都按中断隐指令流程处理。

7.3 寻址方式 7.3-a（14.6 分钟）

- **主旨**：给出寻址方式的总定义，并讲清指令寻址（顺序/跳跃）与数据寻址的指令字格式及立即、直接、隐含三种寻址。
- **关键点**：

1. 寻址方式要解决两件事：确定本条指令的操作数地址，以及下一条要执行指令的指令地址；前者属数据寻址，后者属指令寻址。
2. 指令寻址只有两种：顺序寻址 `(PC)+1→PC`，跳跃寻址由转移指令直接给出新地址。
3. 教材例：0 号单元 LDA 1000、1 号 ADD 1001、2 号 DEC 1200 均为顺序寻址；3 号 JMP 7 使 PC 直接改为 7，即跳跃寻址。
4. 数据寻址的指令字格式为 `操作码 | 寻址特征 | 形式地址A`；形式地址是"指令字中的地址"，有效地址 EA 是"操作数的真实地址"；约定 指令字长 = 存储字长 = 机器字长。
5. 立即寻址：形式地址 A 本身就是操作数，执行阶段不访存。
6. 直接寻址 `EA=A`，有效地址由形式地址直接给出，执行阶段访存一次，且 A 的位数决定了该指令操作数的寻址范围；隐含寻址则把操作数地址藏在操作码中，如 `ADD A` 的另一操作数隐含在 ACC，8086 的 MUL 被乘数隐含在 AX（16 位）或 AL（8 位）、MOVS 源操作数地址隐含在 SI。

- **易错/考点**：形式地址 A 与有效地址 EA 必须分清——直接寻址时二者相等只是特例；直接寻址的寻址范围由 A 的位数决定，不是由机器字长决定。

7.3 寻址方式 7.3-b（17.6 分钟）

- **主旨**：逐个讲完数据寻址的后六种方式：间接、寄存器、寄存器间接、基址、变址，重点是 EA 的求法与访存次数。
- **关键点**：

1. 间接寻址 `EA=(A)`，形式地址给出的是"存放有效地址的单元地址"，一次间址在执行阶段需访存 2 次（先取 EA、再取操作数）。

2. 多次间址靠存储字最高位的标志区分：为 1 表示还需继续间址，为 0 表示已是操作数地址；间接寻址可扩大寻址范围，也便于编制程序（如子程序返回地址）。
3. 寄存器寻址 `EA=R_i`，有效地址就是寄存器编号，操作数在寄存器中，执行阶段不访存。
4. 寄存器间接寻址 `EA=(R_i)`，有效地址在寄存器中、操作数在主存中，执行阶段仍需访存一次，便于编制循环程序。
5. 基址寻址 `EA=(BR)+A`，可用专用基址寄存器，也可用通用寄存器（指令中给出 R_i 作基址寄存器）；面向系统，用于程序重定位，可扩大寻址范围。
6. 变址寻址 `EA=(IX)+A`，IX 为专用变址寄存器（通用寄存器也可充当），面向用户，用于数组、字符串等成批数据处理，同样可扩大寻址范围。

- **易错/考点**：基址与变址的形式完全一样 `EA=(R)+A`，区别在"谁变"——基址寻址中基址寄存器内容由操作系统设定、形式地址 A 作位移；变址寻址中 A 固定不变、变址寄存器内容由用户程序改变。

7.3 寻址方式 7.3-c（11.4 分钟）

- **主旨**：用"求 N 个数平均值"的编程例子对比直接寻址与变址寻址，再讲相对寻址的公式与两个实例。
- **关键点**：

1. 直接寻址版：`LDA D`、`ADD D+1`、`ADD D+2` … `ADD D+(N-1)`、`DIV #N`、`STA ANS`，共 N+2 条指令，指令条数随 N 增长。
2. 变址寻址版：`LDA #0`、`LDX #0`、`ADD X,D`（D 为形式地址）、`INX`（`(X)+1→X`）、`CPX #N`、`BNE M`、`DIV #N`、`STA ANS`，循环体固定，程序长度与 N 无关。
3. 相对寻址 `EA=(PC)+A`，A 是相对于当前指令的位移量，可正可负，用补码表示。
4. 相对寻址例（循环程序）：`M ADD X,D`、`M+1 INX`、`M+2 CPX #N`、`M+3 BNE M` ——转移目标写成"当前 PC + 负位移"，程序整体浮动时无需修改指令。
5. 按字节寻址时位移量以字节计：2000H 处 `JMP *+8` 的目标是 2008H，位移量 8 表示跳过 8 个字节。

- **易错/考点**：算相对位移时基准是"取指后已被修改的 PC"（已加 1 或加指令字节数），不是本条指令的原始地址，这是最容易差一个指令长度的地方。

7.3 寻址方式 7.3-d（9.0 分钟）

- **主旨**：讲最后一种数据寻址——堆栈寻址，包括硬/软堆栈、进栈出栈的 SP 变化与按字节编址时的步长。
- **关键点**：

1. 堆栈分两类：硬堆栈由多个寄存器组成，软堆栈在主存中划出指定的存储空间。
2. 堆栈是先进后出（FILO）、只有一个出入口的结构，栈顶地址由 SP（stack pointer，堆栈指针）指出。

3. 进栈 `(SP)−1→SP` 后再写入，出栈先读出再 `(SP)+1→SP`，SP 始终指向栈顶。
4. 例：ACC=15、SP=200H，`PUSH A` 后 SP=1FFFH、1FFFH 单元存 15；`POP A` 后 ACC=15、SP 恢复为 200H。
5. SP 的修改量与主存编址方式有关：按字编址进栈 `(SP)−1→SP`、出栈 `(SP)+1→SP`；按字节编址且存储字长 16 位时进栈 `(SP)−2→SP`、出栈 `(SP)+2→SP`；32 位时进栈 `(SP)−4→SP`、出栈 `(SP)+4→SP`。

- **易错/考点**：进栈是"先减 SP 再写"、出栈是"先读再增 SP"；按字节编址时步长是"存储字长/8"而非 1，写错方向（-1/+1 弄反）是本题型最常见的失分点。

7.4 指令格式举例（10.1 分钟）

- **主旨**：先列设计指令格式要考虑的因素，再用 IBM 360 的五种格式和 Intel 8086 的变长格式作实例。
- **关键点**：

1. 设计指令格式要考虑：指令系统的兼容性；操作类型（指令个数与操作难易程度）；数据类型（哪些类型可参与操作）；指令格式（字长是否固定、操作码位数、是否用扩展操作码）；地址码位数/个数/寻址方式类型；寄存器个数。
2. IBM 360 的 RR 格式：OP 8 位 + R1 4 位 + R2 4 位，二地址 R-R 型（寄存器-寄存器）。
3. RX 格式：OP 8 + R1 4 + X 4 + B 4 + D 12，二地址 R-M 型，按"基址 + 变址 + 位移"寻址。
4. RS 格式 OP 8 + R1 4 + R2 4 + B 4 + D 12 为三地址 R-M（基址寻址）；SI 格式 OP 8 + I 8 + B 4 + D 12 为立即数-M（基址寻址）；SS 格式 OP 8 + L 8 + B1 4 + D1 12 + B2 4 + D2 12 为二地址 M-M（基址寻址）。
5. Intel 8086 指令字长 1~6 个字节可变：`INC AX` 为 1 字节，`MOV WORD PTR[0204],0138H` 为 6 字节。
6. 8086 按地址格式分类：零地址 `NOP` 1 字节；一地址 `CALL` 段间调用 5 字节、段内调用 3 字节。

- **易错/考点**："几地址"由格式决定而非字长决定——同一台机器可同时有 RR/RX/RS/SI/SS 五种格式；定长指令字（IBM 360）与变长指令字（8086）是另一维度的区别。

7.4 课后习题 7.19 讲解视频（17.9 分钟）

- **主旨**：用一道指令格式设计题，训练"给定位数约束下分配操作码、寄存器号、地址字段"的定量推导。
- **关键点**：

1. 题设：CPU 内有 32 个 32 位通用寄存器，指令系统能容纳 64 种操作，指令字长等于机器字长（32 位）。
2. 位数分配依据：64 种操作需操作码 6 位（`2^6=64`），32 个寄存器需寄存器号 5 位（`2^5=32`）。

3. （1）"寄存器-存储器"型指令必为二地址以上格式：`OP 6 位 | R_i 5 位 | I（间接标志）1 位 | A 20 位`，直接寻址最大空间 `2^20 = 1M` 字。
4. （2）改用基址寻址需在格式中增加 B（基址寻址标志，可省）1 位和 BR_i（基址寄存器号）5 位，形式地址压缩为 14 位：`OP 6 | R_i 5 | I 1 | B 1 | BR_i 5 | A 14`。
5. 基址寻址的最大空间 `2^32 = 4G` 字——有效地址由 32 位基址寄存器内容与位移相加形成，寻址范围只取决于基址寄存器位数，与形式地址 A 的位数无关。

- **易错/考点**：加了基址寄存器字段必然压缩形式地址位宽，但可寻址空间反而从 1M 字扩到 4G 字；判断"寻址范围由谁决定"是本题唯一考点。

7.5 RISC技术（13.4 分钟）

- **主旨**：从 80-20 规律出发解释 RISC 的由来，给出 RISC 与 CISC 的特征清单并逐条对比。
- **关键点**：

1. RISC = Reduced Instruction Set Computer（精简指令集计算机），CISC = Complex Instruction Set Computer（复杂指令集计算机），RISC 是对 CISC 指令系统膨胀的反向修正。
2. 80-20 规律：典型程序中 80% 的语句只用到处理机中 20% 的指令；而执行频度高的简单指令，因复杂指令的存在导致执行速度无法提高。
3. RISC 主要特征：选用频度较高的一些简单指令、复杂功能由简单指令组合；指令长度固定、格式种类少、寻址方式少；只有 LOAD/STORE 指令访存；CPU 中有多个通用寄存器。
4. CISC 主要特征：指令系统复杂庞大、各指令使用频度相差大；指令长度不固定、格式种类多、寻址方式多；访存指令不受限制；CPU 中设专用寄存器；大多数指令需多个时钟周期执行完毕；采用微程序控制器。
5. RISC 更能充分利用 VLSI 芯片面积：控制器用组合逻辑实现、占面积小，省下的面积可放更多寄存器。
6. RISC 更能提高运算速度：指令数、指令格式、寻址方式少，通用寄存器多，采用组合逻辑（硬布线）控制，便于实现指令流水。

- **易错/考点**："RISC 一定比 CISC 快"是错的——RISC 的代价是完成同一功能所需指令条数增多、程序变长，靠流水线补偿；控制方式上 RISC 用组合逻辑、CISC 用微程序，这是最高频的对比点。

第 8 章 CPU 的结构与功能

> 把前面所有部件组装成一台能跑指令的机器：指令周期怎么划分、数据通路怎么走、流水线为什么会冲突、中断怎么响应。

8.1 CPU的结构8.1-b（11.5 分钟）

- **主旨**：按"用户可见 / 不可见"给 CPU 内寄存器分类，并引出控制单元与中断系统。
- **关键点**：
  1. CPU 由 ALU、寄存器、中断系统、CU 四部分组成，通过数据总线、地址总线、控制总线与主存及外设相连。
  2. 用户可见寄存器之通用寄存器：存放操作数，也可作某种寻址方式所需的专用寄存器（如基址、变址）。
  3. 数据寄存器：存放操作数并满足各种数据类型，两个寄存器拼接可存放双倍字长数据。
  4. 地址寄存器：存放地址，其位数应满足最大的地址范围，用于段基值、栈指针等特殊寻址方式。
  5. 条件码寄存器：存放条件码（正、负、零、溢出、进位等），作为程序分支的依据。
  6. 控制和状态寄存器：控制寄存器中 `PC→MAR→M→MDR→IR` 控制 CPU 操作，其中 MAR、MDR、IR 用户不可见而 PC 用户可见；状态寄存器存放条件码；CU 的职责是产生全部指令的微操作命令序列。
- **易错/考点**：可见性判断是送分也是送命题——PC 可见，MAR/MDR/IR 不可见；通用寄存器可见，但"通用"不等于任何指令都能直接寻址它。

8.1 CPU的结构 8.1-a（10.6 分钟）

- **主旨**：开篇交代第 8 章四条主线，并归纳 CPU 中控制器与运算器各自的功能。
- **关键点**：
  1. 第 8 章结构：8.1 CPU 的结构、8.2 指令周期、8.3 指令流水、8.4 中断系统。
  2. 控制器功能清单：取指令、分析指令、执行指令并发出各种操作命令、控制程序输入及结果的输出、总线管理、处理异常情况和特殊请求。
  3. 运算器功能：实现算术运算和逻辑运算。
  4. 控制器功能可归为四类控制：指令控制（取指、分析）、操作控制（发操作命令）、时间控制（时序）、处理中断；运算器对应"数据加工"。
  5. CPU 结构框图从"CPU 与系统总线"入手：CPU 经地址总线、数据总线、控制总线与存储器、I/O 设备交换信息。

- **易错/考点**："取指令、分析指令"属指令控制，"发出操作命令"属操作控制，"处理异常和特殊请求"属处理中断——三类控制不要都塞进"执行"里。

8.2 指令周期8.2-b（10.5 分钟）

- **主旨**：把指令周期拆成四个工作周期，逐个给出其数据流路径。
- **关键点**：

  1. 取指周期数据流：`PC→MAR`→地址总线→存储器；CU 发读命令；数据经数据总线→MDR→IR；同时`(PC)+1→PC`。
  2. 间址周期数据流：IR 中的形式地址字段→MAR→地址总线→存储器，读出的有效地址→MDR。
  3. 执行周期数据流：不同指令的执行周期数据流不同，没有统一模式，需按指令类型分别分析。
  4. 中断周期数据流：保存断点与转入中断服务程序，同样经地址总线、数据总线、控制总线访问存储器。
  5. 四个周期共用同一组总线，因此同一时刻只能有一个周期占用总线，这也是"总线管理"属于控制器功能的原因。

- **易错/考点**：取指周期与间址周期都要访存，区别只在地址来源——取指用 PC，间址用 IR 的形式地址字段；执行周期不一定访存。

8.2 指令周期 8.2-a（14.9 分钟）

- **主旨**：建立指令周期、机器周期、时钟周期的层级概念，并给出指令周期的判断流程与"访存四性质"。
- **关键点**：

  1. 指令周期：取出并执行一条指令所需的全部时间；完成一条指令分取指、分析（合称取指周期）与执行（执行周期）两段。
  2. 每条指令的指令周期不同：`NOP` 只有取指周期，`ADD mem` 需取指周期 + 执行周期。
  3. 具有间接寻址的指令周期 = 取指周期 + 间址周期 + 执行周期；若响应中断还要再加中断周期。
  4. 指令周期流程：取指周期→判"有间址吗"（是则间址周期）→执行周期→判"有中断吗"（是则中断周期）。
  5. CPU 访存有四种性质，正好对应四个工作周期的标志：取指令→取指周期；取地址→间址周期；存取操作数或结果→执行周期；存程序断点→中断周期。

- **易错/考点**：指令周期 ⊃ 机器周期 ⊃ 时钟周期是包含关系而非并列；间址周期只在间接寻址指令中出现，"每条指令都有四个周期"是典型错误说法。

8.3 指令流水8.3.1-a（19.6 分钟）

- **主旨**：从"如何提高机器速度"切入并行性概念，用串行执行与二级流水的对比说明流水的加速原理与失效原因。
- **关键点**：
  1. 提高机器速度三条路：提高访存速度（高速芯片、Cache、多体并行）；提高 I/O 与主机间传送速度（中断、DMA、通道、I/O 处理机、多总线）；提高运算器速度（高速芯片、改进算法）。
  2. 并行有两层含义：并发指两个以上事件在同一时间段发生；同时指两个以上事件在同一时刻发生。
  3. 并行性分级：过程级（程序、进程）与指令级（指令之间）等。
  4. 指令串行执行：取指令1 | 执行指令1 | 取指令2 | 执行指令2...，任一时刻只有一个部件工作，总有一个部件空闲。
  5. 二级流水让取指与执行重叠，理想情况速度提高一倍；但执行时间大于取指时间时，需在取指部件与执行部件之间加指令部件缓冲区。
  6. 条件转移指令破坏流水：必须等上一条指令执行结束才能确定下条指令地址，造成时间损失；六级流水 FI（取指）DI（译码）CO（计算操作数地址）FO（取操作数）EI（执行）WO（写回），理想情况下每 6 个时间单位完成一条、稳态每周期完成一条。
- **易错/考点**：流水线加速比的上限是段数 m，不是无限大；"二级流水速度翻倍"只在取指与执行时间相等且无断流时成立。

8.3 指令流水8.3.1-b（15.3 分钟）

- **主旨**：系统讲影响流水线性能的三类相关——结构相关、数据相关（RAW/WAR/WAW）、控制相关。
- **关键点**：
  1. 结构相关：不同指令争用同一功能部件产生资源冲突，如取指与访存同时要用存储器。
  2. 数据相关：不同指令因重叠操作，可能改变操作数的读/写访问顺序，分写后读、读后写、写后写三种。
  3. 写后读（RAW）：`SUB R1,R2,R3 ;(R2)−(R3)→R1` 与 `ADD R4,R5,R1 ;(R5)+(R1)→R4`，后条要读前条刚写入的 R1。
  4. 读后写（WAR）：`STA M,R2 ;(R2)→M` 与 `ADD R2,R4,R5 ;(R4)+(R5)→R2`，后条写 R2 若早于前条读 R2 即出错。
  5. 写后写（WAW）：`MUL R3,R2,R1 ;(R2)×(R1)→R3` 与 `SUB R3,R4,R5 ;(R4)−(R5)→R3`，两条都写 R3，写回顺序颠倒则结果错。
  6. 控制相关：由转移指令引起，设指令 3 是转移指令，则转移目标未定期间后续指令被取入却作废，形成"转移损失"。

- **易错/考点**：RAW 是真正的数据依赖（必须等前条算完），WAR/WAW 只在乱序执行或重叠写回时才成为问题；判断相关类型看"两条指令对同一寄存器的读/写先后"。

8.3 指令流水 8.3.1-c（9.5 分钟）

- **主旨**：给出流水线三个定量性能指标——吞吐率、加速比、效率的公式与含义。
- **关键点**：
  1. 吞吐率 TP：单位时间内流水线所完成指令或输出结果的数量，分最大吞吐率与实际吞吐率。
  2. 设 m 段流水线各段时间为 Δt，最大吞吐率 `T_pmax = 1/Δt`。
  3. 连续处理 n 条指令的实际吞吐率 `T_p = n / (m·Δt + (n−1)·Δt)`，分母中 m·Δt 是建立（填充）时间。
  4. 加速比 S_p：m 段流水线速度与等功能的非流水线速度之比；流水线完成 n 条指令需 `T = m·Δt + (n−1)·Δt`，等效非流水线需 `T' = n·m·Δt`，故 `S_p = n·m/(m+n−1)`，n→∞ 时趋近 m。
  5. 效率 E：流水线中各功能段的利用率，`E = 各段处于工作时间的时空区 / 各段总的时空区`。
  6. 由于流水线有建立时间和排空时间，各功能段的设备不可能一直处于工作状态，n 越大、m 相对越小，效率越接近 1。
- **易错/考点**：最大吞吐率只看 `1/Δt`，实际吞吐率必须计入填充与排空；加速比上限为段数 m，与指令条数无关地上限——这三条公式的分母最易写错。

8.3 指令流水 8.3.2（15.4 分钟）

- **主旨**：讲流水线的三种多发技术（超标量、超流水、超长指令字）与指令流水线、运算流水线的结构。
- **关键点**：
  1. 超标量（superscalar）技术：每个时钟周期内可并发多条独立指令，配置多个功能部件，靠编译优化静态调度，不能调整指令的执行顺序。
  2. 超流水线（superpipeline）技术：在一个时钟周期内再分段（如 3 段），使一个功能部件在一个时钟周期内被使用多次（3 次）。
  3. 超长指令字（VLIW）技术：由编译程序挖掘指令间潜在的并行性，将多条能并行操作的指令组合成一条，具有多个操作码字段，字长可达几百位。
  4. 指令流水线结构：完成一条指令分 6 段——取指令、指令译码、地址形成、取操作数、操作执行、回写结果，每段需一个时钟周期。
  5. 流水线不出现断流时每 1 个时钟周期出 1 个结果；不采用流水技术则每 6 个时钟周期才出 1 个结果。
  6. 运算流水线：浮点加减运算分对阶、尾数求和、规格化三段，段间用锁存器隔离，分段原则是每段操作时间尽量一致。

- **易错/考点**：超标量是"空间并行"（多套功能部件）、超流水是"时间并行"（同部件分时复用）、VLIW是"编译期静态打包"，三者常被混为一谈；运算流水线分段的依据是"时间均衡"而非"功能划分"。

8.4 中断系统 8.4.1-a（7.8 分钟）

- **主旨**：交代中断的起因、中断系统必须解决的五个问题，以及中断请求标记 INTR 的组织方式。
- **关键点**：
  1. 引起中断的因素：人为设置的中断（如转管指令转入管理程序）、程序性事故（溢出、操作码不能识别、除法非法）等。
  2. 中断系统要解决五个问题：各中断源如何向 CPU 提请求；多个源同时请求怎么办；CPU 什么条件、什么时间、以什么方式响应；如何保护现场；如何寻找入口地址。
  3. 中断请求标记 INTR：一个请求源对应一个 INTR 中断请求标记触发器。
  4. 多个 INTR 组成中断请求标记寄存器，各位分别对应掉电、过热、主存读写校验错、阶上溢、非法除法、键盘输入、打印机输出等中断源。
  5. INTR 分散在各个中断源的接口电路中（也可集中在 CPU 内），CPU 通过查询这些触发器得知谁在请求。
- **易错/考点**：转管指令属"人为设置的中断"，溢出与非法操作码属"程序性事故"，两者都进入中断系统处理，但前者是自愿的、后者是被迫的。

8.4 中断系统 8.4.1-b（12.2 分钟）

- **主旨**：讲中断判优的硬件排队器与软件查询两种实现，以及中断服务程序入口地址的两种寻找方法。
- **关键点**：
  1. 中断判优（优先权仲裁）两种实现：硬件排队器与软件程序查询。
  2. 硬件排队器可分散在各个中断源的接口电路中（链式排队器），也可集中在 CPU 内；链式排队器的优先级由链的顺序固定，INTR1~INTR4 按降序排列。
  3. 软件查询法（程序查询）：按 A、B、C 优先级降序逐个测试"是否 A 请求"，是则转 A 的服务程序入口地址，否则继续测 B。
  4. 硬件向量法寻找入口地址：排队器输出→向量地址形成部件→向量地址。
  5. 向量地址与入口地址不同：12H 单元放 `JMP 200`，则 12H 是向量地址、200 是入口地址；也可在 12H 单元直接放"入口地址 200"，省去一次跳转。
  6. 软件查询法用中断识别程序（入口地址 M），指令如 `SKP DZ 1#`（1# 的 D=0 则跳，D 为完成触发器），逐个跳过未请求的中断源。
- **易错/考点**：向量地址是"存放转移指令或入口地址的单元地址"，入口地址才是服务程序第一条指令的地址——两者差一次访存，考试常要求分别指出。

8.4 中断系统 8.4.2-a（10.4 分钟）

- **主旨**：讲 CPU 响应中断的条件与时机，并拆解中断隐指令完成的三件事。
- **关键点**：
  1. 响应中断的条件：允许中断触发器 EINT=1（已开中断）；CPU 并非在任何条件下都立即响应。
  2. 响应中断的时间：一条指令执行结束时（指令周期最后一个机器周期末）采样中断请求。
  3. 中断隐指令：不是真正的指令、不出现在程序中，由硬件在中断周期自动完成，对程序员透明。
  4. 第一件事：保护程序断点——断点存入特定地址（0 号地址），或断点进栈。
  5. 第二件事：寻找服务程序入口地址——向量地址→PC（硬件向量法），或中断识别程序入口地址 M→PC（软件查询法）。
  6. 第三件事：硬件关中断——把中断标记触发器（INT 的 R-S 触发器）置 0，屏蔽新的中断请求。
- **易错/考点**：不能用"程序里有没有这条指令"判断隐指令；保护断点与关中断由中断隐指令（硬件）完成，而保护寄存器内容由中断服务程序（软件）完成，这条界线必考。

8.4 中断系统 8.4.2-b（17.4 分钟）

- **主旨**：讲保护/恢复现场的分工、多重中断的两个条件，以及屏蔽触发器与屏蔽字的构造。
- **关键点**：
  1. 保护现场含两部分：断点（由中断隐指令完成）与寄存器内容（由中断服务程序完成）。
  2. 恢复现场由中断服务程序完成，与保护现场在服务程序中前后对称。
  3. 多重中断（中断嵌套）：执行某个中断服务程序的过程中，又响应了更高级别的中断请求。
  4. 实现多重中断两个条件：① 提前设置开中断指令（在保护现场之后开中断）；② 优先级高的中断源有权中断优先级低的中断源。
  5. 屏蔽技术用屏蔽触发器 MASK：MASK=0 时未屏蔽，INTR 能被置"1"；MASK=1 时该源请求被封锁，不参与排队。
  6. 屏蔽字按优先级降序排列：16 个中断源中 1 号屏蔽字为全 1，2 号为 `0 1 1…1`，逐级左移一个 0，16 号为 `0…0 1`；即 k 号源屏蔽字前 k-1 位为 0（允许更高级别打断自己），第 k 位及其后为 1。
- **易错/考点**：开中断指令必须放在"保护现场之后"，否则新中断会覆盖尚未保存的现场；屏蔽字中"1"表示屏蔽、"0"表示开放，与"优先级高"不是同一个概念。

8.4 中断系统 8.4.2-c（15.2 分钟）

- **主旨**：用 A/B/C/D 四源的屏蔽字改写实例，说明屏蔽技术如何只改处理优先级而不改响应优先级，并讲多重中断的断点保护。
- **关键点**：

1. 屏蔽技术可改变处理优先级：响应优先级（由硬件排队器决定）不可改变，处理优先级可通过重新设置屏蔽字改变。
2. 原屏蔽字：A=1111、B=0111、C=0011、D=0001，响应优先级与处理优先级均为 A→B→C→D 降序。
3. 新屏蔽字：A=1111、B=0100、C=0110、D=0111，响应优先级不变，处理优先级变为 A→D→C→B 降序。
4. 执行轨迹对比：A、B、C、D 同时请求时，用原屏蔽字按 A、B、C、D 顺序依次处理完毕；用新屏蔽字则 A 处理过程中依次被 D、C、B 打断，形成嵌套。
5. 新屏蔽字的设置顺序：保护现场→置屏蔽字→开中断→中断服务→关中断→恢复现场→恢复屏蔽字→开中断→中断返回。
6. 多重中断的断点保护：断点进栈或断点存入"0"地址，均由中断隐指令完成，中断周期执行 `0→MAR`；用 0 号地址存断点时服务程序需先转存（`LDA 0`、`STA RETURN`），最后用 `JMP @RETURN` 间址返回。

- **易错/考点**：响应优先级由排队器硬件固定、不可改；屏蔽字只改"处理优先级"；且"置屏蔽字"必须在"开中断"之前完成，顺序颠倒会使新屏蔽字来不及生效。

第 9 章 控制单元的功能

> 控制单元是「指挥家」：把每条指令拆成一串微操作，再用机器周期、节拍、脉冲三级时序把它们排好队。

9.1 操作命令的分析 9.1-a（13.7 分钟）

- **主旨**：给出第 4 篇的知识地图，并按工作周期逐个写出微操作命令，先讲取指、间址和非访存类执行周期。
- **关键点**：
  1. 第 4 篇两条线：第 9 章控制单元的功能（9.1 微操作命令分析、9.2 控制单元的功能），第 10 章控制单元的设计（10.1 组合逻辑设计、10.2 微程序设计）。
  2. 完成一条指令分 4 个工作周期：取指周期、间址周期、执行周期、中断周期，每个周期由若干微操作命令组成。
  3. 取指周期微操作：`PC→MAR`→地址线；`1→R`（CU 发读命令）；`M(MAR)→MDR`；`MDR→IR`；`(PC)+1→PC`。
  4. 间址周期微操作：指令的形式地址字段 `Ad(IR)→MAR`；`1→R`；`M(MAR)→MDR`，由此得到有效地址。
  5. 非访存类执行周期：`CLA` 清 A（`0→ACC`）；`COM` 取反（`~ACC→ACC`）。
  6. 移位类执行周期：`SHR` 算术右移（`L(ACC)→R(ACC)`、`ACC0→ACC0`）；`CSL` 循环左移（`R(ACC)→L(ACC)`、`ACC0→ACC0`）。
- **易错/考点**：取指周期中 `(PC)+1→PC` 与读存储器是两条独立微操作，写序列时不能合并；非访存指令的执行周期不含任何访存微操作。

9.1 操作命令的分析 9.1-b（18.7 分钟）

- **主旨**：续讲访存指令、转移指令的微操作序列，汇总三类指令的指令周期结构，最后给出中断周期的微操作。
- **关键点**：
  1. 访存指令 `ADD X`：`Ad(IR)→MAR`、`1→R`、`M(MAR)→MDR`，再由 ALU 完成 `(ACC)+(MDR)→ACC`。
  2. 取数指令 `LDA X` 的完整序列：`Ad(IR)→MAR`；`1→R`；`M(MAR)→MDR`；`MDR→ACC`。
  3. 无条件转移 `JMP X`：`Ad(IR)→PC`，直接把形式地址送入 PC，执行周期不访存。
  4. 条件转移 `BAN X`（负则转）：先判条件码符号位，为负才执行 `Ad(IR)→PC`，否则顺序执行。
  5. 三类指令的周期结构：非访存、直接访存、转移均为"取指 + 执行"；间接访存、间接转移为"取指 + 间址 + 执行"。

6. 中断周期微操作：断点存"0"地址时 `0→MAR`、`1→W`、`PC→MDR`、`MDR→M(MAR)`；断点进栈时改为 `(SP)−1→MAR`；随后 `向量地址→PC`、`0→EINT`（关中断）。

- **易错/考点**：中断周期里的 `1→W` 是写命令，与取指周期的 `1→R` 恰好相反，抄错就变成"把断点读出来"；转移指令执行周期不访存，但间接转移要多一个间址周期。

9.2 控制单元的功能9.2-a（18.8 分钟）

- **主旨**：从 CU 的外特性（输入/输出信号）出发，用 `ADD @ X` 逐周期列出控制信号的产生过程。
- **关键点**：

1. CU 的外特性即其输入输出信号：输入为时钟、指令寄存器 `OP(IR)`、标志、来自系统总线的控制信号；输出为 CPU 内部控制信号与送至系统总线的控制信号。
2. 时钟是 CU 的基准：一个时钟脉冲发一个操作命令，或发一组需同时执行的操作命令。
3. `OP(IR)→CU` 决定该发哪些微操作命令；标志（条件码、进位等）决定条件转移是否成立。
4. CPU 内部控制信号举例：`R_i→R_j`、`(PC)+1→PC`、ALU 的加/减/与/或等。
5. 送至控制总线的信号：`MREQ`（访存控制）、`IO/M`（访 I/O 还是访存）、`RD`（读命令）、`WR`（写命令）、`INTA`（中断响应信号）。
6. 以 `ADD @ X` 为例：取指周期发 PC→MAR、读命令、MDR→IR；间址周期发 `Ad(IR)→MAR` 与读命令取有效地址；执行周期再取操作数并做加法。

- **易错/考点**：`MREQ` / `IO/M` 是"选哪个设备"的选择信号，`RD` / `WR` 是"读还是写"的命令信号，`INTA` 只在中断响应时有效——三者作用层次不同，不能混为一谈。

9.2 控制单元的功能9.2-b（15.4 分钟）

- **主旨**：对比"不采用 CPU 内部总线"与"采用 CPU 内部总线"两种结构下控制信号的写法，重点讲后者。
- **关键点**：

1. 不采用内部总线时各寄存器之间有专用通路，一个微操作只需一个控制信号；采用内部总线后所有寄存器挂在同一条总线上，靠选通信号决定谁送出、谁接收。
2. 取指周期：`PC→MAR` 需 PC 输出选通与 MAR 输入选通同时有效；CU 发读命令；数据线→MDR 需 MDR 输入选通。
3. 间址周期：形式地址→MAR，即 `MDR→MAR`，需 MDR 输出选通与 MAR 输入选通。
4. 执行周期：`MDR→MAR` 取操作数地址、发读命令、数据线→MDR 并送入 Y，再 `MDR→Y→ALU` 完成加法，结果送 AC。
5. 同一时刻总线上只能有一个源输出，故 PC、MDR、Y 等"输出选通"信号必须互斥，否则总线冲突。
6. 采用总线方式连线简洁、易扩展，代价是原本可并行的微操作被迫串行，一条微操作命令对应一组选通信号的组合。

- **易错/考点**：内部总线方式下"源→总线→目的"需要两个控制信号（源出 + 目的入），只写一个数据传不过去，这是本题型最常见的漏项。

9.2 控制单元的功能9.2-c（11.0 分钟）

- **主旨**：讲机器周期、时钟周期（节拍）构成的多级时序系统，以及机器速度与主频的定量关系。
- **关键点**：

  1. 机器周期：所有指令执行过程中的一个基准时间；确定它需考虑每条指令的执行步骤与每一步骤所需时间。
  2. 基准时间的确定：以完成最复杂指令功能的时间为准，或以访问一次存储器的时间为基准；若指令字长 = 存储字长，则取指周期 = 机器周期。
  3. 时钟周期（又称节拍、状态）：把一个机器周期分成若干相等的时间段，是控制计算机操作的最小单位时间。
  4. 一个机器周期内可完成若干个微操作，每个微操作需一定时间，故再分为节拍 T0、T1、T2、T3。
  5. 多级时序：一个指令周期含若干机器周期，一个机器周期含若干时钟周期；如取指周期为一个机器周期，执行周期可含"取有效地址"与"执行"两个机器周期。
  6. 机器速度与主频：主频 f 越快速度越快；在机器周期所含时钟周期数相同的前提下，`MIPS1/MIPS2 = f1/f2`；速度还与机器周期所含时钟周期数、指令周期所含机器周期数有关。

- **易错/考点**："主频高就一定快"不成立——实际速度由主频、每个机器周期的时钟周期数、每条指令的机器周期数三者共同决定，即 CPI 与 f 一起看。

9.2 控制单元的功能9.2-d（8.9 分钟）

- **主旨**：收尾讲 CU 的四种控制方式（同步、异步、联合、人工），核心是"不同微操作命令序列该用什么时序控制方式产生"。
- **关键点**：

  1. 控制方式（control mode）的定义就是"产生不同微操作命令序列所用的时序控制方式"，它决定节拍如何划分、命令如何定时发出。
  2. 同步控制方式（synchronous control）：任一微操作均由统一基准时标的时序信号控制，CLK 的每个时钟周期为一个节拍，节拍序列 T0、T1、T2... 在全机统一。
  3. 定长机器周期方案：一个机器周期固定含 3 个节拍，指令周期由若干机器周期组成，波形上标为"取指令（取有效地址）"与"执行指令"两类机器周期。
  4. 中央控制与局部控制相结合：中央控制节拍宽度一致、覆盖取指与执行全程；局部控制节拍用于等待慢速部件，其节拍宽度与中央控制节拍宽度一致，但只插入在需要的局部区间。
  5. 异步控制方式（asynchronous control）：无基准时标信号、无固定的周期节拍和严格的时钟同步，各部件靠应答（握手）方式衔接。

6. 联合控制方式 = 同步与异步相结合；人工控制方式含三项：Reset、连续/单条指令执行转换开关、符合停机开关。

- **易错/考点**：同步靠统一时钟、异步靠应答，二者最本质的判别标志是"有无基准时标信号"，不要拿"速度快慢"当区分标准。

第 10 章 控制单元的设计

> 同一个控制单元的两条实现路线：硬布线（快但难改）与微程序（慢但规整）。这一章要把微指令格式、编码方式和码点表全部走一遍。

10.1 组合逻辑设计10.1-a（19.4 分钟）

- **主旨**：进入第 10 章控制单元设计，先给出组合逻辑（硬布线）CU 的外特性与框图，再按三条原则把取指、间址的微操作排进节拍。
- **关键点**：
  1. 组合逻辑控制单元（CU）外特性：输入为 IR 的 n 位操作码，经操作码译码产生 2^n 个输出；CLK 经节拍发生器产生 T0...Tn；输出为各控制信号，另有"标志"反馈回 CU。
  2. 节拍信号由 CLK 分频/计数得到，T0 为机器周期的第一个节拍，后续节拍在时钟周期边界上依次推进。
  3. 本节假设：采用同步控制方式，一个机器周期内有 3 个节拍（时钟周期）；CPU 内部结构采用非总线方式（各寄存器间有专用通路）。
  4. 内部结构含 MDR、MAR、PC、IR、AC、ALU、CU，控制信号记为 C0—C11，逐位对应一个微操作命令。
  5. 安排微操作时序三原则：原则一，微操作的先后顺序不得随意更改；原则二，被控对象不同的微操作尽量安排在一个节拍内完成；原则三，占用时间很短的微操作可压在同一节拍内串行完成（如 T1 内的 MDR→IR 与 OP(IR)→ID）。
  6. 取指周期节拍安排：`T0: PC→MAR, 1→R`；`T1: M(MAR)→MDR, (PC)+1→PC`；`T2: MDR→IR, OP(IR)→ID`。
- **易错/考点**：`(PC)+1→PC` 放在 T1 而不是 T2，因为它与 `M(MAR)→MDR` 被控对象不同（PC 与 MDR），可并行，这正是原则二的直接应用。

10.1 组合逻辑设计10.1-b（17.9 分钟）

- **主旨**：把 10 条典型机器指令的执行周期微操作逐条排进 T0/T1/T2 三个节拍，再补上中断周期的节拍安排。
- **关键点**：
  1. 非访存指令（只需 1 个有效节拍）：CLA 为 `T2: 0→AC`；COM 为 `T2: ~AC→AC`（按位取反）；SHR 为 `T2: L(AC)→R(AC), AC0→ACn`（算术右移，符号位保持）；CSL 为 `T2: R(AC)→L(AC), ACn→AC0`（循环左移）；STP 为 `T2: 0→G`（置停机触发器）。
  2. 访存指令统一在 T0 送地址：`T0: Ad(IR)→MAR`，读操作用 `1→R`，写操作用 `1→W`，二者互斥。

3. STA X：`T0: Ad(IR)→MAR, 1→W`；`T1: M(MAR)→MDR, AC→MDR`（同时取旧值与送新值，为写做准备）；`T2: MDR→M(MAR)`。
4. LDA X：`T0: Ad(IR)→MAR, 1→R`；`T1: M(MAR)→MDR`；`T2: MDR→AC`。
5. ADD X：T0 送地址并读，T1 取数到 MDR，T2 完成 `(AC)+(MDR)→AC`；JMP X 为 `T2: Ad(IR)→PC`；BAN X 为"累加器为负则转移"，条件由标志位决定。
6. 中断周期：`T0: 0→MAR, 1→W`（0 号单元作栈指针，同时硬件关中断）；`T1: PC→MDR`（保存断点）；`T2: MDR→M(MAR), 向量地址→PC`（保存断点后转入中断服务程序）。

- **易错/考点**：中断周期 T0 的 `0→MAR` 与 `1→W` 同时给出，是"用 0 号单元暂存断点 + 硬件关中断"两件事并行，容易只记住存断点而漏掉关中断。

10.1 组合逻辑设计10.1-c（15.4 分钟）

- **主旨**：给出组合逻辑设计的完整三步法，并用一张"操作时间表"把 FE/IND/EX 各节拍与各条指令的微操作命令一一对应起来。
- **关键点**：

1. 设计步骤：①列出操作时间表 → ②写出各微操作命令的最简逻辑表达式 → ③按表达式画出逻辑图（本节幻灯片只展开第 1、3 步）。
2. 操作时间表的列结构：工作周期标记（FE 取指 / IND 间址 / EX 执行）、节拍（T0、T1、T2）、状态条件、微操作命令信号，右侧按指令设列（CLA、COM、ADD、STA、LDA、JMP）。
3. FE 行：`T0: PC→MAR, 1→R`；`T1: M(MAR)→MDR, (PC)+1→PC`；`T2: MDR→IR, OP(IR)→ID`；T2 末尾打 `1→IND`、`1→EX` 作为进入下一工作周期的状态标志。
4. IND 行：`T0: Ad(IR)→MAR, 1→R`；`T1: M(MAR)→MDR`；`T2: MDR→Ad(IR)`（间接地址回填 IR），随后 `1→EX`。
5. EX 行按指令打点：`T0: Ad(IR)→MAR` 并 `1→R`（STA 列为 `1→W`）；`T1: M(MAR)→MDR`（STA 为 `AC→MDR`）；`T2` 分别为 `(AC)+(MDR)→AC`（ADD）、`MDR→M(MAR)`（STA）、`MDR→AC`（LDA）、`0→AC`（CLA）。
6. 逻辑图：FE、IND、EX 与 T0—T2 作为与门输入，各指令译码信号（CLA/COM/ADD/STA/LDA/JMP/BAN）再与节拍相与，经或门汇总输出到 `M(MAR)`、`MDR` 等控制线。

- **易错/考点**：同一微操作命令在不同指令下要"或"起来（如 `1→R` 由 FE 的 T0、IND 的 T0、EX 的 T0 三处相或），漏掉任一路都会导致某类指令取数失败。

10.2 微程序设计 10.2.1-a（16.1 分钟）

- **主旨**：提出微程序设计思想——把控制信号固化成"微指令"存进 ROM，用取微指令、执行微指令的方式取代硬布线逻辑。

- **关键点**：

  1. 思想来源：1951 年英国剑桥大学教授 Wilkes 提出微程序设计（microprogramming）概念，把控制逻辑"软件化"。
  2. 层次关系：一条机器指令的功能由若干微操作命令（micro-operation）序列完成；这些命令编成二进制码即微指令（microinstruction），如 `10100000`、`00010010`。
  3. 若干微指令组成一个微程序（microprogram），一条机器指令对应一个微程序，全部微程序固化存入 ROM（控制存储器 CM）。
  4. 控制存储器按地址分区：M、M+1、M+2 为取指周期微程序；其后依次是间址周期微程序、中断周期微程序；P、P+1、P+2 对应 LDA 指令的微程序，K、K+1、K+2 对应 STA 指令的微程序。
  5. 微程序控制单元基本框图：控制存储器 CM、CMAR（控存地址寄存器）、CMDR（控存数据寄存器）、地址译码、微地址形成部件（输入取自 IR 的 OP）、顺序逻辑（输入标志与 CLK）。
  6. 微指令基本格式 = 操作控制字段（产生控制信号）+ 顺序控制字段（决定下一条微指令地址）。

- **易错/考点**：CMAR/CMDR 与主存的 MAR/MDR 是两套寄存器，CMAR 存的是"微指令地址"而非主存地址，混淆是本章最常见的概念错误。

10.2 微程序设计 10.2.1-b（18.4 分钟）

- **主旨**：用 LDA 指令走一遍微程序控制单元的完整工作过程，看清"取微指令—执行微指令—形成下址"的循环。
- **关键点**：

  1. 取微指令的固定三步：`M→CMAR`（送微指令地址）；`CM(CMAR)→CMDR`（读出微指令）；由 CMDR 的操作控制字段发命令，由顺序控制字段给出下址。
  2. 取指微程序第 1 条（地址 M）：操作控制字段产生 `PC→MAR`、`1→R`，下址字段为 `M+1`，微指令形如 `100…0 | M+1`。
  3. 取指微程序第 2 条（地址 M+1）：产生 `M(MAR)→MDR`，下址为 `M+2`，形如 `0100…10 | M+2`；第 3 条完成 `MDR→IR`、`OP(IR)→ID`。
  4. 执行阶段入口由操作码决定：`OP(IR)→微地址形成部件→CMAR`，即把 LDA 微程序首地址 P 送入 CMAR。
  5. LDA 微程序地址 P：产生 `Ad(IR)→MAR`、`1→R`，下址 `P+1`（形如 `0001…0 | P+1`）；地址 P+1：产生 `M(MAR)→MDR`，下址 `P+2`（形如 `0100…0 | P+2`）。
  6. 关键结论：全部微指令事先存在 CM 中，程序执行过程中只需按地址"读出"，不需要像组合逻辑那样在线生成控制信号，因此规整、易改。

- **易错/考点**：`Ad(CMDR)→CMAR` 是"取微指令"的动作（把当前微指令里的下址送 CMAR），与 `Ad(IR)→MAR` 取操作数地址完全是两回事。

10.2 微程序设计 10.2.1-c（19.5 分钟）

- **主旨**：讲微指令的三种编码方式、后续微指令地址的五种形成方法，以及水平型/垂直型两种微指令格式的取舍。
- **关键点**：
  1. 直接编码（直接控制）方式：操作控制字段中每一位直接代表一个微操作命令，一位一线、无需译码，速度最快但字段最长；格式为"操作控制字段 | 下地址"。
  2. 字段直接编码方式：把控制字段分成若干"段"，每段经译码器译码后发出控制信号，段内命令必须互斥，从而用 n 位编码 2^n 个互斥命令、大幅缩短字长。
  3. 字段间接编码方式（隐式编码）：一个字段的译码结果还受另一字段控制，进一步压缩字长但增加译码延迟。
  4. 后续微指令地址的形成：①由微指令的下地址字段直接指出；②根据机器指令操作码经微地址形成部件形成（微程序入口）；③增量计数器 `(CMAR)+1→CMAR`；④分支转移，微指令增设"转移方式 + 转移地址"字段；⑤通过测试网络，把微地址分为非测试地址 h 与测试地址 l，由测试源选择；⑥由硬件直接产生入口地址。
  5. 后续地址形成原理图：OP/IR → 微地址形成部件 → 多路选择 → CMAR，CMDR 的"转移方式/下地址"也送入多路选择，实现多源选一。
  6. 格式比较：水平型微指令一次能定义并执行多个并行操作（可用直接编码、字段直接/间接编码、混合编码），并行能力强、灵活、执行一条机器指令所需微指令数少、速度快，但字长长；垂直型类似机器指令操作码，字短、微程序长、并行性差。水平型是"以短微程序换长微指令"。
- **易错/考点**：字段直接编码的"段"内必须互斥、段间必须相容，若把可能同时有效的两个命令塞进同一段，就会出现命令丢失。

10.2 微程序设计 10.2.2-a（7.7 分钟）

- **主旨**：讲静态/动态微程序设计的区别，以及毫微程序设计（两级控存）和串行/并行微程序控制的结构差异。
- **关键点**：
  1. 静态微程序设计（static microprogramming）：微程序无须改变，控制存储器采用 ROM，成本低、速度快。
  2. 动态微程序设计（dynamic microprogramming）：通过改变微指令和微程序来改变机器指令，便于仿真（emulation）其他机器，控制存储器须采用 EPROM 等可写器件。
  3. 毫微程序设计（nanoprogramming）：设两级控存，第一级存垂直型微指令（MOP），其 MOP 字段作为第二级的毫微指令地址送入 CMAR2。
  4. 结构通路：IR 的 OP 与 `+1`、转移地址 → CMAR1 → 控制存储器（微程序）→ CMDR1（MOP，垂直型）；CMDR1 的 MOP → CMAR2 → 控制存储器（毫微程序）→ CMDR2（水平型微指令）→ 输出控

制信号。

5. 串行微程序控制：取第 i 条微指令 → 执行第 i 条 → 取第 i+1 条 → 执行第 i+1 条，严格顺序，速度慢、控制简单。
6. 并行微程序控制：取第 i 条与执行第 i 条重叠进行，取第 i+1 条与执行第 i+1 条重叠，形成两级流水，速度提高但需处理访存冲突。

- **易错/考点**：毫微程序不是"更细的微程序"，而是用"垂直型微指令 + 水平型毫微指令"两级配合，目的是在保持短字长的同时获得水平型的并行能力。

10.2 微程序设计 10.2.2-b（14.7 分钟）

- **主旨**：以与组合逻辑相同的 CPU 结构为前提，逐条写出 CLA、COM、ADD、STA、JMP 等指令的微操作与节拍，并确定微指令格式。
- **关键点**：

1. 取指阶段微操作分析：`T0: PC→MAR, 1→R`；`T1: M(MAR)→MDR, (PC)+1→PC`；`T2: MDR→IR, OP(IR)→微地址形成部件`。
2. 微程序版本要在每个节拍后补一条"取微指令"动作，因此取指微程序排为：`T0: PC→MAR, 1→R`；`T1: Ad(CMDR)→CMAR`；取指微程序入口地址 M 由微指令下地址字段给出。
3. 非访存指令（如 CLA）：`T0: 0→AC`；`T1: Ad(CMDR)→CMAR`；COM 同理，只是 T0 换成 `~AC→AC`。
4. 访存指令 ADD 需 3 个机器节拍、6 个微指令节拍：`T0 Ad(IR)→MAR, 1→R`；`T1 Ad(CMDR)→CMAR`；`T2 M(MAR)→MDR`；`T3 Ad(CMDR)→CMAR`；`T4 (AC)+(MDR)→AC`；`T5 Ad(CMDR)→CMAR`。
5. STA 结构对称：`T0 Ad(IR)→MAR, 1→W`；`T2 AC→MDR`；`T4 MDR→M(MAR)`，其间 T1、T3、T5 均为 `Ad(CMDR)→CMAR`。
6. 转移类 JMP：`T0: Ad(IR)→PC`；`T1: Ad(CMDR)→CMAR`，地址直接进 PC 而不进 MAR。
7. 微指令格式确定为：编码方式采用直接控制；后续微指令地址由机器指令操作码经微地址形成部件形成。

- **易错/考点**：微程序方案下每个机器节拍被拆成"执行微操作 + 取下一微指令"两步，微指令条数约为组合逻辑节拍数的 2 倍，这是两者最直观的代价差异。

10.2 微程序设计 10.2.2-c（18.1 分钟）

- **主旨**：收尾定量确定微指令字长、讨论省去 CMAR 的优化，并实际编写取指微程序的二进制码点。
- **关键点**：

1. 字长统计：38 条微指令中有 19 条涉及"后续微指令地址→CMAR"，其中 1 条为 `OP(IR)→微地址形成部件→CMAR`，18 条为 `Ad(CMDR)→CMAR`。
2. 优化思路：若让 `Ad(CMDR)` 直接送控制存储器地址线，则省去"输至 CMAR 的时间"，并可省去 CMAR 这一寄存器；同理 `OP(IR)→微地址形成部件` 也直接送控存地址线。
3. 省去 CMAR 后的结构：IR 的 OP → 微地址形成部件 → 多路选择 → 控存地址线；CMDR 的下地址字段也送入多路选择；控制存储器输出送 CMDR，CMDR 的操作控制字段输出控制信号。
4. 码点表结构：列有微程序名称（取指）、微指令地址（八进制 00、01、02…）、操作控制字段（第 0—29 位）、下地址字段。
5. 位定义（直接编码）：第 0 位 `PC→MAR`，第 1 位 `1→R`，第 2 位 `M(MAR)→MDR`，第 3 位 `(PC)+1→PC`，第 4 位 `MDR→IR`，第 10 位 `Ad(IR)→MAR`。
6. 码点填写示例：地址 00 的微指令在 0、1 位上置 1（`PC→MAR`、`1→R`），下地址字段填 01；地址 01 的微指令在 2、3 位上置 1，下地址填 02，如此逐条填满取指微程序。

- **易错/考点**：直接编码下"位号 ↔ 微操作命令"是固定映射，填码点时务必按位定位而不是按顺序数；下地址字段填的是"下一条微指令地址"，不是机器指令地址。
